КАК ДУМАЕТ АРХИТЕКТОР ПРОГРАММНЫХ СИСТЕМ

ОТ БИЗНЕС-ИДЕИ ДО ПРОГРАММНОЙ АРХИТЕКТУРЫ



ПРОЕКТ НЕЛЬЗЯ ОСТАНОВИТЬ.
ЕГО ПРИХОДИТСЯ СОВЕРШЕНСТВОВАТЬ ПРЯМО ВО ВРЕМЯ ДВИЖЕНИЯ.

Оглавление

Предисловие **5**

Часть I. Профессия архитектора **7**

Глава 1. Как читать эту книгу 7

Глава 2. Для кого написана эта книга 11

Глава 3. Как устроена эта книга 17

Глава 4. Принципы книги 22

Глава 5. Архитектурный манифест 28

Часть II. От идеи к требованиям **31**

Глава 6. Что архитектор получает в начале проекта 31

Глава 7. Что делает архитектор, когда думает 36

Глава 8. Почему архитектор не доверяет первому документу 42

Глава 9. Какие вопросы меняют архитектуру системы 48

Глава 10. Размышления автора вслух 55

Глава 11. Как архитектор видит предметную область 59

Часть III . Как архитектор видит систему **67**

Глава 12. Почему сложная система никогда не является единым целым 67

Глава 13. Где проходят настоящие границы системы 73

Глава 14. Почему одинаковые слова могут означать разные вещи . . 80

Глава 15. Как архитектор начинает видеть независимые миры 86

Глава 16. Почему связи важнее компонентов 92

Глава 17. Как рождается архитектурная модель программной системы 99

Глава 18. Когда архитектор впервые рисует схему 106

Часть IV. Архитектура предметной области или как модель становится программой 113

Глава 19. Почему архитектура начинается не с программного обеспечения 113

Глава 20. Где заканчивается предметная область 120

Глава 21. Как предметная область превращается в программную модель 125

Глава 22. Как архитектура начинает говорить на языке бизнеса . . . 130

Глава 23. Что архитектор действительно берет из DDD 135

Часть V. Проектирование программной архитектуры 139

Глава 24. Как архитектор выбирает архитектурный стиль 139

Глава 25. Пример выбора архитектуры приложения 145

Глава 26. Как проектируются программные модули 151

Глава 27. Как проектируются взаимодействия между модулями . . . 157

Глава 28. Как архитектор выбирает технологии 163

Глава 29. Как проектируется инфраструктура системы 169

Глава 30. Как архитектор проектирует развитие системы 175

Предисловие

Разговор архитектора с архитектором

Каждый крупный программный продукт начинается задолго до появления первой строки кода.

Он начинается с идеи.

Затем появляются первые требования, обсуждения с заказчиком, заметки после встреч, диаграммы на доске, таблицы, списки вопросов и многочисленные документы, которые постепенно формируют представление о будущей системе.

В этот момент архитектуры еще не существует.

Существует лишь замысел.

Именно с этого момента начинается работа архитектора.

Его задача состоит не в выборе языка программирования, фреймворка или базы данных. Все это появится значительно позже.

Прежде всего архитектор должен понять предметную область, выявить скрытые требования, обнаружить противоречия, определить границы системы и превратить бизнес-идею в инженерную модель, которую смогут реализовать разработчики.

На практике этот путь редко бывает прямым.

Исходные требования почти всегда оказываются неполными. Одни решения противоречат другим. Некоторые важные детали отсутствуют вовсе. Многие понятия используются без четких определений, а часть вопросов становится очевидной только в процессе анализа.

Именно такая ситуация и стала отправной точкой для этой книги.

В ее основу положена спецификация реального коммерческого проекта. Однако сама книга не посвящена этому проекту.

Исходная спецификация используется лишь как материал для исследования. Она позволяет пройти весь путь, который ежедневно проходит архитектор: от первого знакомства с требованиями до построения законченной архитектуры сложной информационной системы.

На протяжении книги мы не будем искать единственно правильные ответы.

Архитектура редко допускает абсолютные решения.

Каждое архитектурное решение существует в контексте конкретных ограничений: бизнеса, бюджета, сроков, команды, законодательства, масштабируемости, безопасности и множества других факторов.

Поэтому цель этой книги заключается не в том, чтобы предложить готовые рецепты.

Ее цель — показать процесс инженерного мышления.

Мы будем анализировать требования, задавать вопросы, искать противоречия, сравнивать альтернативы, оценивать последствия принятых решений и постепенно формировать архитектуру системы так, как это происходит в реальной профессиональной практике.

В книге намеренно отсутствуют универсальные правила и категоричные утверждения.

В инженерии почти не существует решений, одинаково подходящих для всех проектов.

Поэтому каждое решение будет рассматриваться вместе с причинами его появления, областью применения, преимуществами, ограничениями и возможными альтернативами.

Эта книга не является описанием конкретной технологии.

Она посвящена процессу проектирования сложных корпоративных информационных систем.

Все рассматриваемые технологии, инструменты и архитектурные подходы служат лишь средствами достижения цели. Главным объектом исследования остается архитектурное мышление — способность превращать неопределенность требований в последовательную, логичную и реализуемую инженерную систему.

Если после прочтения этой книги читатель начнет иначе смотреть на требования, чаще задавать вопросы заказчику и принимать архитектурные решения более осознанно, значит ее главная задача будет выполнена.

Часть I. Профессия архитектора

Эта часть посвящена не технологиям, а профессии архитектора, его мышлению и подходу к проектированию сложных информационных систем.

Глава 1. Как читать эту книгу

Читать эту книгу можно по-разному..

Для одних читателей она станет последовательным руководством, позволяющим шаг за шагом пройти путь от первых требований заказчика до построения архитектуры сложной корпоративной системы. Другие будут открывать только те главы, которые необходимы для решения конкретной задачи: проектирования модели данных, разработки системы авторизации, выбора архитектурного стиля или анализа требований.

Оба подхода являются правильными.

Книга изначально проектировалась таким образом, чтобы поддерживать каждый из них.

Цель книги

Главная цель этой книги — не познакомить читателя с очередной технологией и не предложить универсальный рецепт построения программных систем.

Таких рецептов не существует.

Цель книги шире.

Она заключается в том, чтобы показать процесс проектирования сложной информационной системы глазами архитектора.

В профессиональной практике архитектор редко начинает работу с выбора технологий. До обсуждения языка программирования, базы данных или инфраструктуры ему необходимо понять предметную область, сформулировать бизнес-задачи, выявить ограничения, определить ключевые сущности системы и только после этого принимать инженерные решения.

Именно этот путь последовательно рассматривается на протяжении всей книги.

Особое внимание уделяется не только конечным результатам, но и самому процессу принятия решений. Мы будем анализировать требования, задавать вопросы, сравнивать альтернативы, обсуждать последствия каждого выбора и оценивать, как отдельные решения влияют на архитектуру системы в целом.

Особенности структуры

Большинство технических книг построено по принципу последовательного изложения материала: каждая следующая глава опирается на предыдущую.

В данной книге используется иной подход.

С одной стороны, все главы объединены общей логикой проектирования и образуют единую историю создания корпоративной информационной системы.

С другой стороны, каждая глава представляет собой самостоятельный законченный материал.

Это позволяет обращаться к книге как к профессиональному руководству, открывая только те разделы, которые необходимы в текущий момент работы.

При этом все главы связаны между собой перекрестными ссылками и образуют единую систему знаний.

Линейное и выборочное чтение

Если читатель впервые знакомится с архитектурным проектированием крупных информационных систем, рекомендуется читать книгу последовательно.

Такой подход позволяет увидеть взаимосвязь между анализом требований, моделированием предметной области, проектированием архитектуры, разработкой API, построением модели данных и выбором инфраструктурных решений.

Однако книга допускает и другой способ работы.

Практикующий архитектор редко изучает архитектуру «целиком». Обычно перед ним стоит конкретная задача: разработать систему разграничения доступа, спроектировать механизм аудита, определить структуру базы данных или выбрать способ интеграции нескольких сервисов.

В подобных случаях книга может использоваться как самостоятельный источник информации по отдельной теме.

Каждая глава написана таким образом, чтобы ее можно было читать независимо от остальных разделов.

Использование книги как справочника

Со временем большинство читателей, вероятно, перестанут читать книгу последовательно.

Она превратится в рабочий инструмент.

Во время проектирования новой системы архитектор сможет обращаться к книге как к справочнику по вопросам мышления системного архитектора, чтобы осмыслить задачу, проверить свои рассуждения и найти ориентиры для принятия проектных решений.

По этой причине многие главы содержат повторяющиеся элементы.

Это сделано намеренно.

Каждая тема рассматривается как самостоятельная инженерная задача и включает описание проблемы, анализ требований, возможные архитектурные решения, сравнение альтернатив, обсуждение компромиссов и итоговое решение.

Такой формат позволяет работать с любой главой независимо от остальных.

Как работать с архитектурными решениями

Одной из особенностей книги является отсутствие категоричных рекомендаций.

В профессиональной литературе нередко встречаются утверждения вида «правильно делать именно так».

Практика показывает, что подобные рекомендации редко выдерживают столкновение с реальными проектами.

Каждая информационная система создается в условиях собственных ограничений.

Размер команды, сроки разработки, бюджет, законодательные требования, ожидаемая нагрузка, уровень компетенции разработчиков и десятки других факторов непосредственно влияют на архитектуру будущего продукта.

Поэтому каждое архитектурное решение, рассматриваемое в книге, сопровождается анализом контекста, преимуществ, недостатков, альтернативных подходов и возможных последствий.

Цель состоит не в том, чтобы предложить готовые ответы.

Гораздо важнее показать способ мышления, позволяющий самостоятельно принимать инженерные решения в условиях неопределенности.

Вместо заключения

Если после прочтения очередной главы читатель не согласится с предложенным архитектурным решением, это не будет означать, что книга оказалась бесполезной.

Напротив.

Архитектура развивается благодаря профессиональной дискуссии.

Любое решение может быть поставлено под сомнение, если существуют аргументы, подтверждающие более удачный подход.

Поэтому я предлагаю относиться к этой книге не как к сборнику готовых решений, а как к приглашению к профессиональному диалогу.

Если в процессе чтения вы будете мысленно спорить с автором, искать собственные аргументы и сравнивать описанные подходы со своим опытом, значит книга выполняет свою главную задачу.

Теперь вы знаете, как архитектор пришел к этому решению. А если вы нашли лучшее решение — значит книга достигла своей цели.

Глава 2. Для кого написана эта книга

Каждый профессиональный труд имеет своего читателя

Невозможно написать книгу, одинаково полезную для всех.

Одни книги помогают сделать первые шаги в профессии. Другие посвящены изучению конкретной технологии. Третьи описывают практический опыт решения отдельных инженерных задач.

Эта книга занимает особое место.

Она посвящена не программированию как таковому и не отдельным технологиям. Ее предмет — проектирование сложных корпоративных информационных систем и процесс принятия архитектурных решений.

Поэтому прежде чем продолжить чтение, важно понять, кому адресована эта книга и какой уровень подготовки она предполагает.

Целевая аудитория

Книга адресована специалистам, которые занимаются проектированием, развитием и сопровождением сложных информационных систем. Независимо от официального названия должности, всех их объединяет одна задача — принятие технических решений, влияющих на архитектуру программного продукта.

В первую очередь книга будет полезна следующим специалистам.

Архитектор программного обеспечения (Software Architect)

Специалист, отвечающий за проектирование архитектуры отдельной программной системы.

В сферу его ответственности входит определение внутренней структуры приложения, выбор архитектурных решений, обеспечение масштабируемости, надежности, безопасности и возможности дальнейшего развития продукта.

Архитектор решений (Solution Architect)

Специалист, проектирующий комплексные решения, объединяющие несколько взаимосвязанных информационных систем.

Он определяет границы отдельных компонентов, способы их взаимодействия, интеграцию с внешними сервисами и обеспечивает целостность архитектуры на уровне всего решения.

Корпоративный архитектор (Enterprise Architect)

Специалист, отвечающий за архитектуру информационного ландшафта организации в целом.

Его работа включает разработку архитектурных стандартов, формирование технологической стратегии предприятия, управление развитием информационных систем и обеспечение их соответствия долгосрочным целям бизнеса.

Технический руководитель команды (Technical Lead)

Разработчик, который помимо реализации функциональности принимает технические и архитектурные решения внутри команды.

Он отвечает за качество программного кода, организацию разработки, проведение технических ревью, наставничество и согласование архитектурных решений с другими командами.

Ведущий разработчик серверной части (Lead Backend Engineer)

Инженер, отвечающий за архитектуру серверной части приложения.

Как правило, именно он принимает решения о построении прикладного интерфейса (API), модели данных, интеграции сервисов, производительности, безопасности и взаимодействии между компонентами системы.

Старший разработчик серверной части (Senior Backend Engineer)

Опытный инженер, который стремится выйти за рамки реализации отдельных задач и понимать систему как единое целое.

Такие специалисты постепенно переходят от написания программного кода к принятию архитектурных решений и часто становятся техническими руководителями или архитекторами.

Технический директор небольших и средних компаний (Chief Technology Officer, CTO)

Руководитель, определяющий техническую стратегию компании и принимающий непосредственное участие в проектировании программных продуктов.

В небольших организациях технический директор нередко совмещает обязанности архитектора, технического руководителя и ведущего разработчика. Именно поэтому вопросы, рассматриваемые в книге, являются частью его ежедневной профессиональной деятельности.

Разработчики, готовящиеся к переходу в архитектурную роль

Отдельную категорию составляют инженеры, обладающие значительным опытом разработки и стремящиеся перейти к проектированию информационных систем.

Для них книга может стать практическим руководством, позволяющим понять, чем мышление архитектора отличается от мышления разработчика и какие вопросы необходимо научиться задавать еще до начала проектирования.

Независимо от должности, всех специалистов, для которых написана эта книга, объединяет одна общая особенность.

Они перестали воспринимать программное обеспечение как совокупность отдельных файлов, классов или сервисов. Их основной объект внимания — система в целом, ее развитие, устойчивость, масштабируемость и способность решать бизнес-задачи.

Именно такой взгляд отличает архитектора от разработчика и лежит в основе всех последующих глав этой книги.

Требуемый уровень подготовки

Книга предполагает, что читатель уже обладает практическим опытом разработки программного обеспечения.

Не имеет принципиального значения, на каком языке программирования построена ваша профессиональная деятельность. Архитектурные принципы значительно шире конкретных технологий и сохраняют свою актуальность независимо от используемых инструментов.

Предполагается, что вы уже участвовали в разработке программных продуктов, принимали технические решения и сталкивались с вопросами, на которые невозможно ответить, открыв документацию по языку программирования или фреймворку.

Если вам приходилось задаваться вопросами:

- Почему система устроена именно так?
- Можно ли было принять другое архитектурное решение?
- Какие последствия будет иметь выбранный подход через несколько лет?
- Почему одна и та же задача решается совершенно по-разному в разных проектах?

значит эта книга написана именно для вас.

Что книга предполагает известным

Чтобы сосредоточиться на вопросах архитектуры, автор сознательно не рассматривает основы разработки программного обеспечения.

Предполагается, что читатель уже знаком с:

- принципами объектно-ориентированного программирования;
- построением клиент-серверных приложений;
- протоколом HTTP и принципами передачи данных;
- архитектурным стилем REST;
- прикладными интерфейсами (API);

- реляционными базами данных и языком SQL;
- системами контроля версий;
- контейнеризацией приложений;
- базовыми принципами тестирования программного обеспечения.

Это не означает, что перечисленные темы не будут встречаться в книге.

Наоборот, они будут использоваться постоянно.

Однако внимание будет сосредоточено не на их устройстве, а на роли, которую они играют при проектировании архитектуры программных систем.

Чего в книге нет

Чтобы избежать ложных ожиданий, важно сразу обозначить границы книги.

Она не является учебником по языкам программирования, фреймворкам или отдельным технологиям.

Здесь вы не найдете пошаговых инструкций по настройке среды разработки, установке программного обеспечения, созданию пользовательского интерфейса или написанию программного кода.

Эта книга также не предлагает универсальных рецептов построения архитектуры.

В инженерной практике практически не существует решений, одинаково подходящих для всех проектов.

Каждая система создается в условиях собственных ограничений: предметной области, требований бизнеса, бюджета, сроков, особенностей команды и множества других факторов.

Поэтому архитектурные решения рассматриваются не как готовые шаблоны для копирования, а как результат анализа требований, оценки альтернатив и поиска разумных компромиссов.

Главная цель книги — не дать готовые ответы, а показать ход инженерного мышления, который позволяет находить собственные решения в условиях реальных проектов.

Вместо заключения

Профессиональный рост разработчика редко заканчивается освоением нового языка программирования или очередного фреймворка.

Со временем приходит понимание, что успех проекта определяется не количеством используемых технологий, а качеством принятых инженерных решений.

Именно в этот момент разработчик начинает мыслить как архитектор.

Если, открывая новую задачу, вы все чаще спрашиваете себя не «как это реализовать?», а «почему система должна быть устроена именно так?», значит вы уже находитесь на этом пути.

Эта книга написана для тех, кто хочет научиться находить ответы именно на такие вопросы.

Глава 3. Как устроена эта книга

Книга как архитектурный проект

Каждая сложная система начинается с архитектуры.

Это справедливо для программного обеспечения, информационных систем, предприятий и даже городов.

Книга, которую вы держите в руках, не стала исключением.

Она также была спроектирована.

Ее структура не является случайной последовательностью глав.

Каждая часть занимает свое место, каждая глава решает определенную задачу, а все вместе они образуют единую систему знаний, построенную по тем же принципам, по которым создаются современные корпоративные информационные системы.

Поэтому прежде чем перейти к вопросам архитектуры программного обеспечения, полезно понять архитектуру самой книги.

Общая структура

Материал организован таким образом, чтобы повторять естественный ход работы архитектора.

В реальном проекте архитектор никогда не начинает работу с выбора базы данных, языка программирования или технологии обмена сообщениями.

Он начинает с понимания бизнеса.

Именно поэтому книга развивается в той же последовательности, в которой развивается любой серьезный проект.

Сначала рассматривается профессия архитектора и его роль в процессе разработки.

Затем начинается анализ требований и изучение предметной области.

После этого строится доменная модель, принимаются архитектурные решения, проектируется модель данных, рассматриваются вопросы безопасности, интеграции и инфраструктуры.

Таким образом каждая следующая часть книги естественным образом продолжает предыдущую.

Это не последовательность технологий.

Это последовательность инженерного мышления.

Логика перехода между частями

Одной из распространенных ошибок при изучении архитектуры является восприятие отдельных технологий как независимых знаний.

На практике этого не происходит.

Каждое архитектурное решение становится следствием предыдущих решений.

Модель предметной области определяет структуру приложения.

Архитектура приложения влияет на организацию данных.

Модель данных определяет способы интеграции.

Интеграция влияет на требования к инфраструктуре.

Поэтому книга построена по принципу причинно-следственной связи.

Каждая новая часть отвечает на вопрос, который неизбежно возникает после изучения предыдущей.

Например.

После анализа требований возникает вопрос:

Как представить предметную область?

После построения предметной модели появляется следующий вопрос:

Как разделить систему на компоненты?

После выбора архитектурного стиля возникает вопрос:

Как организовать хранение данных?

Затем появляются вопросы безопасности, интеграции, эксплуатации и развития системы.

Именно таким образом развивается не только книга.

Именно так развивается большинство реальных проектов.

Связь между главами

Несмотря на последовательную структуру книги, каждая глава представляет собой самостоятельное исследование конкретной архитектурной задачи.

Главы можно читать независимо друг от друга.

Однако они не являются изолированными.

Каждая новая тема использует выводы, полученные в предыдущих главах.

Например, невозможно полноценно обсуждать систему разграничения доступа, не определив роли пользователей.

Невозможно проектировать API, не понимая структуры предметной области.

Невозможно говорить о масштабировании, не имея представления об общей архитектуре системы.

Поэтому каждая глава одновременно является самостоятельной и входит в состав единой архитектурной модели книги.

Перекрестные ссылки

В процессе работы архитектор постоянно возвращается к уже принятым решениям.

Изменение требований может потребовать пересмотра модели данных.

Новая архитектура безопасности способна изменить структуру API.

Появление дополнительных бизнес-требований может привести к пересмотру первоначальной декомпозиции системы.

Поэтому книга построена как взаимосвязанная система.

Во многих главах будут встречаться ссылки на ранее рассмотренные темы.

Это сделано намеренно.

Архитектура не существует в виде набора независимых решений.

Она представляет собой единую сеть взаимосвязанных зависимостей.

Именно поэтому изменение одной части системы почти всегда отражается на других ее компонентах.

Архитектура книги

Если посмотреть на книгу как на инженерный проект, можно заметить, что она обладает собственной архитектурой.

Части и главы книги последовательно раскрывают основные этапы работы архитектора.

Профессия архитектора ↓ От идеи к требованиям ↓ Как архитектор видит систему ↓ Архитектура предметной области или как модель становится программой ↓ Проектирование программной архитектуры

↓ Развитие и сопровождение архитектуры

Именно в такой последовательности обычно развивается крупный проект.

Поэтому книга построена не вокруг технологий.

Она построена вокруг процесса принятия архитектурных решений.

Это означает, что практически любая глава отвечает не только на вопрос «что необходимо сделать?», но и на значительно более важный вопрос:

«Почему именно в этот момент появляется необходимость принять такое решение?»

Именно эта логика объединяет всю книгу в единое целое.

Вместо заключения

При работе над крупными информационными системами архитектор постепенно перестает воспринимать проект как набор отдельных компонентов.

Со временем появляется способность видеть систему целиком, понимать взаимосвязь между ее элементами и прогнозировать последствия принимаемых решений.

Эта книга построена по тем же принципам.

Каждая глава является самостоятельной, однако полностью раскрывает свой смысл только как часть общей архитектуры книги.

Поэтому наиболее ценным результатом чтения станет не запоминание отдельных архитектурных решений.

Гораздо важнее увидеть взаимосвязи между ними.

Именно способность видеть систему целиком отличает архитектора от специалиста, хорошо знающего отдельные технологии.

Глава 4. Принципы книги

Любая инженерная дисциплина опирается на систему принципов.

Они не описывают конкретные технологии, не предлагают готовых решений и не определяют порядок выполнения работы.

Их назначение значительно глубже.

Принципы формируют способ мышления специалиста.

Именно они позволяют принимать решения в ситуациях, когда готовых ответов не существует.

Архитектура программного обеспечения относится именно к таким дисциплинам.

Практически каждый проект уникален. Предметная область, требования бизнеса, состав команды, сроки разработки, бюджет, законодательные ограничения и ожидаемая нагрузка делают невозможным применение универсальных рецептов.

Поэтому в основе этой книги лежит не набор технологий, а система принципов, которые будут последовательно использоваться при анализе каждой архитектурной задачи.

Все последующие главы являются практическим продолжением этих принципов.

Архитектура начинается с диалога с бизнесом

Архитектура программного обеспечения существует не сама по себе.

Ее назначение — помочь бизнесу решать конкретные задачи.

Именно поэтому работа архитектора начинается не с выбора языка программирования, базы данных или облачной платформы.

Первым объектом исследования всегда становится бизнес.

Необходимо понять, какую проблему должна решить система, какие процессы она автоматизирует, кто будет ею пользоваться, какие

ограничения существуют и каким образом будет измеряться успешность проекта.

Только после этого появляются требования к программному обеспечению.

Попытка проектировать архитектуру без понимания бизнеса неизбежно приводит к ситуации, когда технически качественная система оказывается бесполезной для заказчика.

Поэтому на страницах этой книги практически любое техническое решение будет начинаться с анализа бизнес-контекста.

Архитектура является продолжением бизнеса, а не самостоятельной целью разработки.

Требования важнее технологий

Современная индустрия предлагает огромное количество технологий, инструментов и архитектурных подходов.

Каждый год появляются новые языки программирования, базы данных, фреймворки и платформы.

Однако технологии остаются лишь средствами реализации.

Они никогда не являются отправной точкой проектирования.

Архитектор сначала отвечает на вопрос «что необходимо построить?», и только затем — «с помощью каких средств это реализовать?»

В профессиональной практике одна и та же задача может быть успешно реализована совершенно разными технологиями.

И наоборот, использование самых современных инструментов не гарантирует успех проекта, если требования поняты неверно.

Поэтому в книге технологии рассматриваются не как самоцель, а как инструменты, выбор которых определяется требованиями конкретной системы.

Каждое решение должно быть обосновано

Архитектура отличается от программирования тем, что большинство принимаемых решений оказывают влияние на систему в течение многих лет.

Поэтому выбор архитектурного решения не может строиться на личных предпочтениях, модных тенденциях или привычках команды.

Каждое решение должно иметь обоснование.

Архитектор обязан понимать:

- какую проблему решает выбранный подход;
- почему были отвергнуты альтернативные варианты;
- какие преимущества дает принятое решение;
- какие ограничения оно накладывает;
- каким образом оно повлияет на дальнейшее развитие системы.

Если на эти вопросы невозможно дать аргументированный ответ, значит решение принято преждевременно или требует дополнительного анализа.

В книге каждое архитектурное решение будет сопровождаться объяснением причин его появления, а не только описанием конечного результата.

Любое решение имеет цену

В инженерии не существует бесплатных решений.

Каждое преимущество неизбежно сопровождается определенными издержками.

Повышение надежности увеличивает сложность системы.

Рост производительности может привести к удорожанию инфраструктуры.

Гибкость архитектуры нередко требует дополнительных усилий при разработке.

Даже самое удачное решение всегда представляет собой компромисс между несколькими противоречивыми требованиями.

Именно поэтому архитектор оценивает не только преимущества, но и цену, которую придется заплатить за их получение.

В этой книге практически каждая архитектурная альтернатива будет анализироваться через призму компромиссов.

Вопрос будет звучать не «какое решение лучше?», а «какое решение наиболее соответствует условиям данного проекта?»

Не существует универсальных решений

Одной из наиболее распространенных ошибок является поиск единственно правильной архитектуры.

Практика показывает обратное.

Решение, успешно работающее в одном проекте, может оказаться совершенно непригодным в другом.

Даже проекты, принадлежащие к одной предметной области, могут существенно различаться по масштабу, требованиям к надежности, скорости разработки, объему данных, организационной структуре команд и многим другим параметрам.

Поэтому архитектура всегда зависит от контекста.

Все рассматриваемые в книге решения будут анализироваться применительно к конкретным условиям.

При изменении исходных условий изменится и архитектурное решение.

Архитектура постоянно развивается

Информационная система никогда не остается неизменной.

Изменяются требования бизнеса.

Появляются новые пользователи.

Растут объемы данных.

Увеличивается нагрузка.

Изменяются законодательные требования.

Развиваются технологии.

Все это приводит к необходимости постоянного развития архитектуры.

Поэтому задача архитектора заключается не в создании идеальной системы, а в проектировании архитектуры, способной адаптироваться к изменениям.

Хорошая архитектура — это не архитектура, в которой ничего не меняется.

Хорошая архитектура позволяет изменять систему без разрушения ее целостности.

Именно поэтому вопросы эволюции архитектуры будут сопровождать читателя на протяжении всей книги.

Архитектор отвечает за последствия своих решений

Архитектурные решения отличаются от большинства технических решений длительным жизненным циклом.

Последствия выбора базы данных, модели авторизации, структуры предметной области или архитектурного стиля могут проявляться спустя годы после начала эксплуатации системы.

Поэтому ответственность архитектора не заканчивается утверждением диаграмм или написанием технической документации.

Она продолжается на протяжении всей жизни проекта.

Архитектор отвечает не только за то, чтобы система была построена.

Он отвечает за то, чтобы она могла развиваться, сопровождаться, масштабироваться и оставаться понятной для следующих поколений инженеров.

Именно способность прогнозировать долгосрочные последствия собственных решений отличает архитектора от специалиста, принимающего локальные технические решения.

Вместо заключения

На протяжении всей книги мы будем неоднократно возвращаться к этим принципам.

Каждая новая тема — будь то моделирование предметной области, проектирование системы разграничения доступа, выбор архитектурного стиля или организация взаимодействия между сервисами — будет рассматриваться через их призму.

Именно они станут критерием оценки архитектурных решений.

Технологии неизбежно изменятся.

Появятся новые инструменты, новые языки программирования и новые подходы к разработке.

Однако принципы инженерного мышления остаются значительно более устойчивыми.

Именно поэтому основу книги составляют не технологии, а принципы, позволяющие принимать обоснованные архитектурные решения независимо от того, какие инструменты будут использоваться завтра.

Глава 5. Архитектурный манифест

Архитектура начинается не с технологий.

Она начинается с понимания бизнеса.

Хорошая архитектура не демонстрирует сложность.

Она делает сложную систему понятной.

Архитектор принимает решения не ради настоящего.

Он проектирует будущее системы.

Каждое архитектурное решение имеет цену.

Бесплатных решений не существует.

Нет правильной архитектуры.

Есть архитектура, соответствующая поставленной задаче.

Любая система однажды изменится.

Поэтому хорошая архитектура проектируется для изменений.

Архитектор отвечает не только за то, что построено сегодня.

Он отвечает за последствия своих решений через годы.

Прежде чем выбирать технологии, необходимо понять требования.

Прежде чем писать код, необходимо понять предметную область.

Прежде чем проектировать систему, необходимо понять бизнес.

Архитектура — это искусство поиска разумного компромисса.

Хороший архитектор знает множество технологий.

Выдающийся архитектор знает, когда их не следует использовать.

Программный код можно переписать.

Архитектурные ошибки переписываются значительно тяжелее.

Сложность нельзя уничтожить.

Ее можно только правильно распределить.

Идеальная архитектура не существует.

Существует архитектура, наиболее подходящая для конкретного проекта.

Архитектор не предсказывает будущее.

Он делает систему готовой к его наступлению.

Архитектура существует для бизнеса.

Не наоборот.

Часть II. От идеи к требованиям

Глава 6. Что архитектор получает в начале проекта

Миф о техническом задании

Существует распространенное представление о том, как начинается работа архитектора.

Заказчик завершает анализ требований, готовит подробное техническое задание, согласовывает его со всеми заинтересованными сторонами и передает архитектору документ, содержащий всю необходимую информацию для проектирования системы.

После этого архитектор внимательно изучает документ, выбирает технологии, строит архитектуру и передает проект команде разработки.

Подобный сценарий существует.

Однако в реальной практике он встречается значительно реже, чем принято считать.

Большинство проектов начинается совершенно иначе.

Первое знакомство с проектом

В действительности архитектор почти никогда не получает полностью сформированное техническое задание.

Вместо него ему передают некоторый набор материалов, отражающих текущее понимание проекта.

Это могут быть:

- документ в формате Word или PDF;
- презентация для инвесторов;
- коммерческое предложение;
- результаты интервью с заказчиком;
- протоколы встреч;

- переписка по электронной почте;
- сообщения в корпоративном мессенджере;
- схемы, нарисованные на доске;
- диаграммы из Miro или FigJam;
- описание существующего бизнес-процесса;
- ссылки на аналогичные системы;
- действующий программный продукт, который необходимо переработать;
- устное описание идеи во время первой встречи.

Иногда архитектор получает сразу несколько подобных источников.

Иногда — только один.

Нередко они противоречат друг другу.

Информации всегда недостаточно

Практически любой первый документ обладает одной общей особенностью.

Он не отвечает на все вопросы.

Наоборот.

После первого знакомства с проектом вопросов становится значительно больше.

Заказчик рассказывает, что необходимо построить.

Архитектор начинает понимать, чего в этом описании нет.

Именно в этот момент становится заметной разница между мышлением заказчика и мышлением архитектора.

Заказчик описывает желаемый результат.

Архитектор начинает искать недостающие элементы будущей системы.

Техническое задание — не истина

Одна из наиболее опасных ошибок начинающего архитектора заключается в восприятии технического задания как завершенного и абсолютно достоверного документа.

На практике техническое задание представляет собой лишь одну из версий понимания проекта на определенный момент времени.

Любой подобный документ содержит:

- предположения;
- упрощения;
- недосказанности;
- противоречия;
- устаревшую информацию;
- требования, которые еще будут изменены.

Поэтому задача архитектора заключается не в безусловном выполнении технического задания.

Его задача — понять реальную систему, которая скрывается за этим документом.

Именно поэтому опытный архитектор никогда не начинает проектирование сразу после первого прочтения требований.

Что на самом деле получает архитектор

Если посмотреть глубже, становится очевидно, что архитектор получает не техническое задание.

Он получает значительно более сложный объект.

Он получает неопределенность.

Именно неопределенность становится отправной точкой любого архитектурного проекта.

Архитектору предстоит превратить разрозненные идеи, пожелания, ограничения, предположения и ожидания различных участников проекта в логически непротиворечивую модель будущей системы.

Эта работа требует не столько знания технологий, сколько умения анализировать информацию, задавать вопросы и выявлять взаимосвязи между отдельными требованиями.

Именно поэтому архитектура начинается значительно раньше выбора языка программирования или базы данных.

Она начинается с понимания задачи.

Источники требований

На протяжении проекта архитектор получает информацию из множества источников.

Каждый из них описывает систему с собственной точки зрения.

Заказчик говорит о бизнесе.

Будущие пользователи — о повседневной работе.

Руководители подразделений — об организационных процессах.

Юристы — о требованиях законодательства.

Специалисты по информационной безопасности — об ограничениях доступа.

Эксплуатационные подразделения — о надежности, резервировании и сопровождении.

Разработчики существующей системы — о накопленных технических проблемах.

Ни один источник не обладает полной картиной.

Только объединение этих точек зрения позволяет постепенно сформировать целостное представление о будущем продукте.

Первый документ — это начало исследования

Одной из наиболее важных особенностей архитектурной профессии является отношение к исходной документации.

Для аналитика она может быть результатом проделанной работы.

Для разработчика — основанием для реализации функциональности.

Для архитектора — это начало исследования.

Каждый новый документ порождает новые вопросы.

Каждый ответ меняет понимание системы.

Каждое изменение требований способно повлиять на архитектуру.

Именно поэтому опытный архитектор воспринимает первый документ не как руководство к действию, а как гипотезу, которую еще предстоит проверить.

Вместо заключения

Работа архитектора начинается не с построения диаграмм и не с выбора технологий.

Она начинается с анализа информации.

Первый документ, попадающий к архитектору, редко содержит готовые ответы.

Его главная ценность заключается в другом.

Он позволяет начать задавать правильные вопросы.

Именно способность видеть за текстом документа будущую систему отличает архитектора от большинства других участников проекта.

Все последующие главы этой части будут посвящены тому, как из первоначальной неопределенности постепенно рождается архитектура.

Архитектор не получает требования. Он получает информацию различного качества, из которой ему предстоит сформировать требования.

Глава 7. Что делает архитектор, когда думает

Архитектор не начинает с решений

Получив описание нового проекта, начинающий разработчик часто сразу начинает искать решение.

Он размышляет о языке программирования, базе данных, архитектурном стиле, способах интеграции и других технических вопросах.

Архитектор действует иначе.

Прежде чем принять хотя бы одно техническое решение, он пытается понять саму задачу.

На этом этапе в его голове еще нет микросервисов, монолита, Kubernetes или очередей сообщений.

Есть только множество вопросов и неопределенность.

Главная задача архитектора — не придумать решение как можно быстрее.

Главная задача — понять проблему настолько глубоко, чтобы правильное решение стало логическим следствием этого понимания.

Архитектор строит модель происходящего

Первое, что начинает делать архитектор, — мысленно строить модель будущей системы.

Пока еще нет диаграмм.

Нет классов.

Нет таблиц базы данных.

Есть только попытка ответить на несколько фундаментальных вопросов.

Что представляет собой этот бизнес?

Кто является участниками процесса?

Какие события происходят внутри системы?

Какие действия совершают пользователи?

Что является целью каждого процесса?

Какие ограничения уже существуют?

Архитектор постепенно превращает разрозненную информацию в единую картину.

Это похоже на сборку мозаики, когда отдельные элементы еще не соединены между собой, но они возможно уже лежат на столе в этот момент..

Архитектор ищет неизвестное

После первого знакомства с проектом опытный архитектор редко говорит:

«Теперь мне все понятно.»

Гораздо чаще он произносит совсем другую фразу:

«Я пока понимаю далеко не все.»

Это не проявление неуверенности.

Это профессиональный подход.

Архитектор сознательно ищет то, чего пока не знает.

Неопределенность представляет для него значительно больший интерес, чем уже известные факты.

Каждый неизвестный элемент становится потенциальным источником будущих архитектурных ошибок.

Поэтому хороший архитектор постепенно уменьшает область неизвестного, прежде чем приступит к проектированию системы.

Архитектор ищет противоречия

Полученные материалы редко образуют непротиворечивую картину.

Одни документы говорят одно.

Другие — другое.

Бизнес хочет ускорить вывод продукта на рынок.

Юридический отдел требует соблюдения законодательства.

Специалисты по информационной безопасности предлагают дополнительные механизмы защиты.

Будущие пользователи ожидают максимально простой интерфейс.

Эксплуатационные подразделения стремятся минимизировать стоимость сопровождения.

Каждое из этих требований само по себе может быть правильным.

Однако одновременно выполнить их все в максимальном объеме зачастую невозможно.

Поэтому архитектор начинает искать не только требования.

Он начинает искать противоречия между ними.

Именно они в дальнейшем будут определять архитектуру системы.

Архитектор расставляет приоритеты

Одной из важнейших особенностей архитектурного мышления является способность определять относительную значимость требований.

На практике невозможно построить систему, одинаково хорошо удовлетворяющую всем пожеланиям заинтересованных сторон.

Возникает необходимость выбора.

Что важнее для данного проекта:

скорость разработки или надежность?

стоимость инфраструктуры или производительность?

простота сопровождения или гибкость?

удобство пользователей или усиленные требования безопасности?

требования бизнеса или требования законодательства?

Ответов, одинаково подходящих для всех проектов, не существует.

Приоритеты определяются целями конкретной системы.

Для банковского программного обеспечения соблюдение нормативных требований может быть важнее скорости вывода новых функций.

Для стартапа, наоборот, решающим фактором может стать время выхода продукта на рынок.

Для медицинской информационной системы безопасность данных становится обязательным условием независимо от стоимости реализации.

Архитектор не просто анализирует требования.

Он определяет их иерархию.

Именно эта иерархия впоследствии становится основой большинства архитектурных решений.

Архитектор ищет взаимосвязи

Любое требование существует не изолированно.

Оно связано с другими требованиями, пользователями, бизнес-процессами и ограничениями.

Изменение одного элемента почти всегда влияет на множество остальных.

Поэтому архитектор постепенно начинает видеть не отдельные требования, а сеть взаимосвязей между ними.

Именно здесь появляется системное мышление.

Будущая архитектура рождается не из списка функций.

Она рождается из понимания того, каким образом все элементы системы влияют друг на друга.

Архитектор задает вопросы самому себе

Прежде чем встретиться с заказчиком, архитектор начинает внутренний диалог.

Он задает себе вопросы, ответы на которые определят дальнейший ход анализа.

Что является главной целью системы?

Какие бизнес-процессы являются ключевыми?

Какие участники принимают решения?

Что произойдет, если одно из требований окажется неверным?

Какие ограничения нельзя нарушить ни при каких обстоятельствах?

Какие требования можно изменить без существенного влияния на систему?

Каждый подобный вопрос постепенно уменьшает область неопределенности.

Именно так рождается понимание предметной области.

Архитектор сознательно откладывает принятие решений

Одной из наиболее сложных профессиональных привычек является способность не торопиться с выводами.

Чем опытнее архитектор, тем осторожнее он относится к первым решениям.

Он понимает, что преждевременное решение почти всегда строится на неполной информации.

Поэтому архитектор сознательно сопротивляется желанию немедленно выбрать технологии или предложить архитектурную схему.

Сначала необходимо понять задачу.

И только после этого появляется право принимать технические решения.

Что происходит в голове архитектора

Если попытаться описать архитектурное мышление одной фразой, то оно выглядит следующим образом.

Архитектор непрерывно отвечает самому себе на четыре вопроса.

Что я уже знаю?

Чего я пока не знаю?

Что противоречит друг другу?

Что окажет наибольшее влияние на архитектуру системы?

Пока ответы на эти вопросы не найдены, проектирование еще не началось.

Все предыдущие действия являются подготовкой к нему.

Вместо заключения

Со стороны может показаться, что архитектор в этот момент ничего не делает.

Он не пишет код.

Не рисует диаграммы.

Не обсуждает технологии.

Однако именно в эти часы и дни выполняется самая важная часть архитектурной работы.

Будущая система рождается не тогда, когда появляется первая схема.

Она рождается значительно раньше — в тот момент, когда архитектор начинает понимать проблему, видеть взаимосвязи, обнаруживать противоречия и определять, какие решения действительно имеют значение.

Именно поэтому архитектура начинается не с инструментов и не с технологий.

Она начинается с мышления.

Глава 8. Почему архитектор не доверяет первому документу

Первый документ никогда не является всей системой

Получив первое описание проекта, неопытный специалист часто воспринимает его как законченный источник информации.

Возникает естественное желание как можно быстрее перейти к проектированию.

Кажется, что все необходимое уже изложено в документе, остается лишь выбрать технологии и приступить к реализации.

Архитектор мыслит иначе.

Для него первый документ — это не описание системы.

Это лишь первое представление о ней.

Он понимает, что любой документ создается людьми, которые видят проект со своей точки зрения.

Поэтому архитектор доверяет документу как источнику информации, но не рассматривает его как полное описание будущей системы.

Документ отражает взгляд автора

Любой документ имеет автора.

А значит — имеет точку зрения.

Коммерческий директор будет говорить о прибыли и развитии бизнеса.

Руководитель проекта — о сроках и бюджете.

Аналитик — о функциональных требованиях.

Юрист — о соблюдении законодательства.

Специалист по информационной безопасности — о защите информации.

Будущие пользователи — об удобстве ежедневной работы.

Каждый из них описывает одну и ту же систему.

Но каждый описывает только ту часть, которую считает наиболее важной.

Поэтому архитектор всегда задает себе вопрос:

Кто написал этот документ и какую часть картины он показывает?

Не существует полного технического задания

Одним из самых устойчивых мифов разработки является представление о существовании идеального технического задания.

Документа, в котором описано абсолютно все.

Практика показывает обратное.

Даже самые подробные документы не способны заранее предусмотреть все особенности будущей системы.

По мере развития проекта появляются новые требования.

Изменяются бизнес-процессы.

Возникают законодательные ограничения.

Меняются приоритеты компании.

Появляются новые заинтересованные стороны.

Если архитектор рассматривает техническое задание как окончательную истину, архитектура очень быстро перестает соответствовать реальному проекту.

Поэтому опытный архитектор воспринимает требования как развивающуюся модель, а не как неизменный документ.

Самые важные требования часто отсутствуют

На практике наиболее критичные требования нередко вообще не попадают в первые документы.

Обычно в них подробно описываются функции системы.

Но почти ничего не говорится о характеристиках, без которых система не сможет работать.

Например:

Какое количество пользователей предполагается?

Как быстро должна работать система?

Какие требования предъявляются к доступности?

Какие существуют ограничения законодательства?

Как долго необходимо хранить данные?

Какие действия должны фиксироваться в журнале аудита?

Какие требования предъявляются к информационной безопасности?

Как будет происходить развитие системы через несколько лет?

Именно эти вопросы впоследствии оказывают наибольшее влияние на архитектуру.

Архитектор ищет то, чего нет

Во время чтения документа внимание архитектора постепенно смещается.

Он начинает анализировать не только написанное.

Не меньший интерес представляет отсутствующая информация.

Какие процессы описаны слишком поверхностно?

Какие участники системы вообще не упоминаются?

Какие ограничения не сформулированы?

Какие события отсутствуют в сценариях?

Какие данные нигде не описаны?

Именно отсутствие информации чаще всего становится причиной наиболее серьезных архитектурных ошибок.

Поэтому хороший архитектор учится видеть не только текст документа, но и возникающие вокруг него пустоты.

Любое требование требует проверки

Даже если требование сформулировано совершенно однозначно, архитектор не спешит принимать его без дополнительного анализа.

Он пытается понять причины его появления.

Какую проблему бизнеса решает данное требование?

Что произойдет, если оно будет реализовано иначе?

Кто является инициатором этого требования?

Останется ли оно актуальным через несколько лет?

Не противоречит ли оно другим требованиям?

Проверка требований не означает недоверие к заказчику.

Она означает стремление построить систему, которая действительно будет решать поставленные задачи.

Архитектор ищет скрытые требования

Во многих проектах самые важные требования никогда не произносятся вслух.

Заказчик считает их очевидными.

Команда предполагает, что они известны всем участникам проекта.

В результате они вообще не попадают в документацию.

Например:

система должна работать круглосуточно;

данные не должны теряться;

операции должны быть обратимыми;

действия пользователей необходимо журналировать;

важные изменения требуют согласования;

персональные данные должны быть защищены.

Подобные требования редко становятся предметом обсуждения.

Но именно они зачастую определяют архитектуру значительно сильнее, чем функциональные возможности системы.

Недоверие — это профессиональная ответственность

Со стороны может показаться, что архитектор постоянно сомневается в полученной информации.

На самом деле он сомневается не в людях.

Он сомневается в полноте собственного понимания системы.

Это принципиальная разница.

Архитектор понимает, что каждый участник проекта обладает только частью общей картины.

Его задача состоит в том, чтобы объединить эти части в единую модель.

Именно поэтому профессиональное сомнение становится одним из основных инструментов архитектурного анализа.

Вместо заключения

Архитектор работает не с документами. Он работает с моделями реальности.

Сама система существует независимо от документа.

Первый документ открывает проект.

Но он никогда его не завершает.

Каждая новая встреча.

Каждый новый вопрос.

Каждое уточнение.

Каждый найденный бизнес-процесс постепенно меняют представление архитектора о системе.

Поэтому хороший архитектор начинает проектирование не тогда, когда получает первое техническое задание.

Он начинает его тогда, когда понимает ограничения первого документа.

Именно способность видеть за документом реальную систему отличает архитектора от человека, который просто внимательно читает требования.

Поэтому задача архитектора заключается не в том, чтобы безукоризненно реализовать написанный текст, а в том, чтобы понять реальный бизнес, реальные процессы и реальные ограничения, которые этот текст пытается описать.

Глава 9. Какие вопросы меняют архитектуру системы

Архитектура рождается из вопросов

После изучения первых материалов архитектор редко приступает к проектированию.

Наоборот.

Именно в этот момент начинается новый этап работы.

Этап вопросов.

Со стороны может показаться, что архитектор просто уточняет детали проекта.

Однако опыт показывает, что правильно поставленный вопрос способен изменить архитектуру значительно сильнее, чем десятки последующих технических решений.

Поэтому вопросы архитектора — это не способ заполнить пробелы в документации.

Это инструмент исследования будущей системы.

Вопросы рождаются из неопределенности

После анализа первых документов архитектор почти никогда не обладает полной картиной проекта.

Он видит предположения.

Замечает противоречия.

Находит отсутствующие связи.

Именно в этих местах начинают появляться вопросы.

Важно понимать, что архитектор не составляет список вопросов заранее.

Каждый следующий вопрос рождается как следствие предыдущего ответа.

Таким образом постепенно исследуется вся предметная область.

Архитектор не движется по заранее подготовленному сценарию.

Он движется вслед за логикой системы.

Хороший вопрос отличается от обычного

Не каждый вопрос имеет одинаковую ценность.

Например, вопрос:

Сколько пользователей будет работать в системе?

дает полезную информацию.

Но значительно важнее другой вопрос:

Что произойдет, если количество пользователей увеличится в десять раз?

Первый вопрос описывает текущее состояние.

Второй заставляет задуматься о будущем.

Именно такие вопросы меняют архитектурные решения.

Архитектор всегда стремится задавать вопросы, которые раскрывают причины, ограничения и последствия.

Архитектор ищет причины, а не ответы

Опыт показывает, что многие требования являются следствием других требований.

Например, заказчик говорит:

Необходимо реализовать возможность удаления документов.

Начинающий специалист начинает обсуждать способы удаления.

Архитектор задает другой вопрос:

Почему документы вообще необходимо удалять?

Ответ может полностью изменить архитектуру.

Возможно, документы запрещено удалять законодательством.

Возможно, требуется только скрыть их из пользовательского интерфейса.

Возможно, необходимо вести полный аудит всех изменений.

Возможно, удаление связано с ошибками пользователей.

Один вопрос способен превратить простую операцию удаления в совершенно другую модель хранения данных.

Некоторые вопросы меняют всю систему

Существуют вопросы, после которых архитектура проекта начинает выглядеть совершенно иначе.

Например:

Что является самым ценным активом системы?

Что произойдет, если сервис станет недоступен на один час?

Какие ошибки недопустимы ни при каких обстоятельствах?

Какие данные запрещено потерять?

Какие действия невозможно отменить?

Какие процессы являются критически важными для бизнеса?

Ответы на подобные вопросы определяют не отдельные функции.

Они определяют фундаментальные свойства всей системы.

Архитектор задает неудобные вопросы

Иногда лучшие вопросы оказываются самыми неприятными.

Что произойдет, если основной поставщик перестанет существовать?

Кто несет ответственность за ошибочные действия пользователя?

Что делать, если два администратора одновременно изменяют одни и те же данные?

Как компания продолжит работать после полного отказа центра обработки данных?

Как будут выглядеть бизнес-процессы через пять лет?

Подобные вопросы редко нравятся участникам проекта.

Они заставляют обсуждать риски, о которых не хочется думать.

Однако именно эти вопросы позволяют избежать архитектурных ошибок, исправление которых впоследствии может стоить очень дорого.

Архитектор задает вопросы самому себе

Самые важные вопросы архитектор задает еще до встречи с заказчиком.

Что я считаю очевидным без достаточных оснований?

Какие предположения я сделал автоматически?

Что может оказаться неверным?

Какие решения я уже мысленно принял слишком рано?

Какие требования пока существуют только в моей интерпретации?

Такая внутренняя проверка помогает избежать одной из самых опасных ошибок — проектирования системы на основе собственных предположений.

Вопросы помогают определить приоритеты

Во время предыдущей главы мы говорили о том, что архитектор постоянно определяет приоритеты требований.

Однако сами приоритеты невозможно определить без вопросов.

Что важнее для бизнеса — скорость вывода новой функциональности или надежность системы?

Какой простой допустим?

Какие требования обязательны с точки зрения законодательства?

Какими возможностями пользователи действительно будут пользоваться ежедневно?

От каких функций можно отказаться, если сроки окажутся ограниченными?

Каждый подобный вопрос постепенно формирует архитектурные приоритеты проекта.

Именно поэтому вопросы становятся инструментом принятия решений, а не просто способом получения информации.

Самый важный вопрос архитектора

За годы работы у каждого архитектора появляется множество профессиональных вопросов.

Однако один из них сопровождает практически любой проект.

Почему?

Почему бизнес хочет именно это решение?

Почему процесс устроен именно так?

Почему существует данное ограничение?

Почему правило нельзя изменить?

Почему эта проблема вообще возникла?

До тех пор пока архитектор не понимает причины, он не может быть уверен, что нашел правильное решение.

Очень часто оказывается, что заказчик предлагает не проблему, а уже придуманное им решение.

Задавая вопрос «Почему?», архитектор возвращается к первопричине и получает возможность найти более удачное архитектурное решение.

Вопросы продолжаются всю жизнь проекта

Распространено мнение, что вопросы задаются только на этапе анализа требований.

Практика показывает обратное.

Архитектор задает вопросы на протяжении всей жизни системы.

Во время проектирования.

Во время разработки.

Во время тестирования.

Во время эксплуатации.

После каждого серьезного инцидента.

После появления новых требований.

После изменения законодательства.

Архитектура развивается вместе с проектом.

А значит, развивается и процесс поиска правильных вопросов.

Вместо заключения

Профессионализм архитектора определяется не количеством ответов, которые он знает.

Его определяет качество вопросов, которые он способен задать.

Хороший вопрос заставляет участников проекта по-новому взглянуть на проблему.

Позволяет обнаружить скрытые ограничения.

Выявляет противоречия.

Помогает определить настоящие приоритеты.

Иногда один своевременно заданный вопрос способен изменить архитектуру всей системы еще до начала разработки.

Именно поэтому опытный архитектор никогда не боится признаться, что чего-то не знает.

Гораздо опаснее ситуация, когда архитектор перестает задавать вопросы.

Теперь становится ясно, каким инструментом он уменьшает неопределенность.

Не диаграммами.

Не технологиями.

Не моделями данных.

Вопросами.

Именно поэтому способность задавать вопросы — это не навык коммуникации. Это один из важнейших инструментов архитектурного мышления. Он превращает хаотичный набор пожеланий в осмысленную модель системы, на основе которой уже можно принимать инженерные решения. Это естественно подводит нас к следующей главе, где мы покажем, как из ответов на эти вопросы рождаются уже не предположения, а формализованные требования.

Глава 10. Размышления автора вслух

Сейчас возникла мысль, которую почти нигде не встречал в книгах по архитектуре.

Потому что архитекторы часто говорят:

"Нужно устранить противоречия."

Но в реальном Enterprise-проекте это почти никогда невозможно.

Наоборот.

Противоречия — это естественное состояние любого большого проекта.

Бизнес хочет одно.

Безопасность — другое.

Юристы — третье.

Эксплуатация — четвертое.

Разработчики — пятое.

Финансы — шестое.

Все они правы одновременно.

И архитектор прекрасно понимает, что невозможно сделать систему, которая на 100% удовлетворит всех.

Тогда что происходит на самом деле?

Архитектор не устраняет противоречия.

Он делает их явными.

Затем обсуждает их со всеми заинтересованными сторонами.

После этого стороны принимают решение:

чем готовы пожертвовать ради достижения более важной цели.

То есть рождается архитектурный компромисс.

И вот этот компромисс уже фиксируется как архитектурное решение.

Мне кажется, это намного глубже.

Например.

Бизнес говорит

Нужно вывести систему через три месяца.

Информационная безопасность отвечает

Тогда мы не успеем реализовать все механизмы защиты.

Разработка говорит

Тогда придется отказаться от части функциональности.

Юристы говорят

Но требования закона обязательны.

Что произошло?

Никто не ошибается.

Все требования остаются.

Но теперь появляется компромисс.

Например.

Первая версия будет содержать только обязательную функциональность.

или

Дополнительные проверки безопасности будут реализованы во второй версии.

или

Запуск переносится на один месяц.

Требования никуда не исчезли.

Противоречие тоже осталось.

Но стороны приняли решение.

Архитектор вообще не занимается требованиями бизнеса.

Он занимается управлением противоречиями между требованиями.

Это совершенно другой взгляд на профессию.

Требования редко становятся полностью непротиворечивыми.

В сложных проектах это скорее исключение, чем правило.

Новые бизнес-цели, законодательные ограничения, требования информационной безопасности, пожелания пользователей, ограничения бюджета и сроки реализации постоянно вступают друг с другом в конфликт.

Задача архитектора состоит не в том, чтобы устранить эти противоречия. Чаще всего это невозможно.

Его задача — сделать их явными, показать последствия каждого варианта решения, помочь заинтересованным сторонам принять осознанный компромисс и зафиксировать достигнутые договоренности как основу будущей архитектуры.

Именно после этого требования становятся не идеальными, а достаточно согласованными для начала проектирования.

А вот здесь, как мне кажется, появляется одно из лучших определений архитектуры, которое я могу считать авторским.

Не такое:

Архитектура — это структура программной системы.

Не такое:

Архитектура — это набор компонентов и связей.

А значительно глубже.

Архитектура — это материализованный компромисс между противоречивыми требованиями различных заинтересованных сторон.

Потрясающе, насколько много в этой фразе.

Она объясняет практически всё.

Почему существуют разные архитектуры?

Потому что разные проекты принимают разные компромиссы.

Почему нет идеальной архитектуры?

Потому что не существует проекта без противоречивых требований.

Почему два опытных архитектора могут предложить разные решения одной задачи?

Потому что они по-разному расставили приоритеты и приняли разные компромиссы.

Мне кажется, я сейчас выразил одну из центральных идей всей книги. Я бы хотел, чтобы читатели запомнили эту мысль и сами проверили её на практике. Тогда читатель постепенно придет к очень зрелому пониманию профессии: архитектор — это не человек, который знает все технологии и всегда находит идеальное решение. Архитектор — это человек, который умеет находить наилучший возможный компромисс в условиях, когда идеального решения не существует.

И именно поэтому архитектурные решения нельзя оценивать в отрыве от тех компромиссов, которые привели к их появлению. Эта мысль, на мой взгляд, отражает реальную инженерную практику гораздо точнее, чем привычные определения архитектуры, и может стать одной из ключевых идей этой книги.

Глава 11. Как архитектор видит предметную область

Архитектор перестает видеть документы

До этого момента архитектор работал с требованиями, вопросами и ответами.

Он изучал документы.

Проводил встречи.

Выявлял противоречия.

Уточнял ограничения.

Постепенно уменьшал область неопределенности.

Но наступает момент, когда документы перестают быть главным источником информации.

В сознании архитектора начинает возникать нечто гораздо более важное.

Он начинает видеть сам бизнес.

Не его описание.

Не текст технического задания.

Не диаграммы.

А процессы, которые существуют независимо от документов.

Именно в этот момент начинается настоящее архитектурное мышление.

Предметная область существует независимо от информационной системы

Очень распространенная ошибка — считать, что предметная область рождается вместе с программным обеспечением.

Это не так.

Банк существовал задолго до появления банковских информационных систем.

Больницы лечили пациентов задолго до появления электронных медицинских карт.

Логистические компании перевозили грузы задолго до появления цифровых платформ.

Нефтегазовые компании продавали оборудование, заключали контракты и организовывали поставки задолго до того, как появились специализированные маркетплейсы.

Информационная система не создает предметную область.

Она лишь помогает автоматизировать уже существующие процессы.

Поэтому архитектор изучает не программу.

Он изучает мир, который программа должна отражать.

Архитектор ищет не функции, а процессы

Начинающий разработчик часто воспринимает систему как набор экранов.

Каталог.

Карточка товара.

Корзина.

Форма регистрации.

Личный кабинет.

Архитектор видит совершенно другое.

Он задает вопросы:

Что происходит до регистрации?

Что происходит после публикации объявления?

Почему оборудование проходит инспекцию?

Кто принимает решение о покупке?

Какие события запускают следующий этап процесса?

Перед ним постепенно возникает не интерфейс.

Перед ним возникает жизнь организации.

Именно процессы становятся фундаментом будущей архитектуры.

За каждым действием стоит причина

Предметная область состоит не из операций.

Она состоит из причин, по которым эти операции выполняются.

Почему объявление проходит модерацию?

Почему подписка имеет срок действия?

Почему инспекция обязательна для определенного оборудования?

Почему один пользователь может выполнять несколько ролей?

Почему некоторые действия требуют подтверждения?

Каждый ответ раскрывает не функцию системы, а бизнес-правило.

Именно бизнес-правила формируют предметную область.

Архитектор постепенно начинает видеть систему как совокупность правил, по которым живет организация.

Архитектор начинает замечать закономерности

После анализа большого количества информации отдельные факты перестают существовать изолированно.

Возникают повторяющиеся модели.

Некоторые процессы выглядят очень похожими.

Различные участники совершают одинаковые действия.

Одни и те же правила применяются в разных местах системы.

Архитектор начинает видеть закономерности.

Именно благодаря этому сложная система постепенно становится понятной.

Хаос начинает превращаться в структуру.

Архитектор ищет границы

По мере изучения предметной области становится очевидно, что далеко не все процессы тесно связаны между собой.

Одни процессы образуют естественные группы.

Другие почти не взаимодействуют.

Например, управление подписками живет по своим правилам.

Логистика — по своим.

Публикация объявлений — по своим.

Финансовые расчеты — по своим.

Это еще не архитектура системы.

Это естественные границы самой предметной области.

Архитектор не придумывает их.

Он их обнаруживает.

Именно позже эти границы могут стать основой для выделения подсистем, сервисов или отдельных модулей.

Архитектор замечает язык бизнеса

Во время общения разные участники проекта начинают использовать одни и те же слова.

Объявление.

Контракт.

Инспекция.

Подписка.

Представитель.

Комиссия.

Статус.

На первый взгляд это обычные термины.

Но архитектор замечает, что разные люди иногда вкладывают в них разный смысл.

Для продавца «публикация» означает размещение объявления.

Для модератора — завершение проверки.

Для покупателя — появление объявления в каталоге.

Получается, что одно слово скрывает несколько различных понятий.

Именно здесь архитектор впервые понимает, насколько важен единый язык проекта.

Эта мысль станет отправной точкой следующей главы.

Архитектор перестает видеть программу

После того как предметная область становится понятной, архитектор начинает смотреть на будущую систему совершенно иначе.

Он уже не думает о формах ввода.

Не представляет таблицы базы данных.

Не выбирает фреймворки.

Перед ним возникает модель предметной области.

Информационная система становится лишь одним из способов реализовать эту модель.

Это один из самых важных моментов профессионального роста.

Хороший архитектор проектирует не приложение.

Он проектирует цифровое отражение реального бизнеса.

Вместо заключения

Предметная область не находится в техническом задании.

Она существует в деятельности организации.

В решениях людей.

В бизнес-процессах.

В правилах.

В ограничениях.

В целях.

Документы лишь помогают увидеть эту реальность.

Но не заменяют ее.

Именно поэтому архитектор сначала изучает предметную область, а уже потом начинает проектировать программную систему.

Потому что невозможно правильно спроектировать отражение того, что еще не удалось понять.

Мне кажется, именно здесь начинается настоящая архитектура

До этой главы архитектор учился понимать проект.

С этой главы он начинает видеть его внутреннее устройство.

И это очень важный переход.

Потому что дальше мы уже будем говорить не о сборе требований, а о моделировании предметной области, выделении сущностей, поиске границ, построении единого языка и принятии архитектурных решений.

И я бы предложил одну очень важную мысль, которая может стать основой не только следующей главы, но и всей третьей части книги:

Архитектор не изобретает предметную область. Он открывает ее, как исследователь открывает неизвестную территорию.

Мне кажется, эта фраза очень точно описывает профессию. Архитектор не придумывает правила бизнеса, не навязывает ему структуру и не создает мир с нуля. Его задача — внимательно исследовать уже существующую реальность, понять ее внутренние закономерности и только затем создать систему, которая будет этой реальности соответствовать. Именно такой подход, на мой взгляд,

отличает зрелое архитектурное мышление от простого проектирования программного обеспечения.

Часть III . Как архитектор видит систему

Глава 12. Почему сложная система никогда не является единым целым

Архитектор никогда не видит всю систему сразу

После того как предметная область становится понятной, перед архитектором возникает новая проблема.

Теперь он знает, что должна делать система.

Но одновременно он начинает понимать и другое.

Система оказывается настолько большой, что удержать ее целиком в сознании практически невозможно.

В ней десятки бизнес-процессов.

Сотни бизнес-правил.

Тысячи взаимосвязей.

Десятки различных участников.

Множество исключений.

Если попытаться представить все это одновременно, мышление начинает терять ясность.

Это происходит не потому, что архитектор недостаточно опытен.

Так вероятно устроен человеческий мозг.

Человеческое мышление имеет границы

Человек способен анализировать сложные вещи.

Но существует предел, после которого количество информации начинает мешать пониманию.

Когда элементов становится слишком много, внимание рассеивается.

Одни взаимосвязи начинают скрывать другие.

Причины смешиваются со следствиями.

Исключения заслоняют общие правила.

В результате система перестает восприниматься как единое целое.

Она превращается в хаотичный набор фактов.

Это не проблема программирования.

Это фундаментальное ограничение человеческого мышления.

Именно поэтому архитектор никогда не пытается осмыслить большую систему сразу.

Архитектор начинает упрощать мир

Слово «упрощать» часто понимают неправильно.

Архитектор не делает систему проще.

Он делает ее понятнее.

Это разные вещи.

Представьте карту мира.

На ней невозможно одновременно показать каждую улицу, каждый дом, каждое дерево и каждый автомобиль.

Такая карта станет бесполезной.

Поэтому картограф строит разные карты.

Политическую.

Физическую.

Геологическую.

Транспортную.

Каждая из них показывает один и тот же мир.

Но каждая делает это по-своему.

Архитектор поступает точно так же.

Он не изменяет систему.

Он выбирает тот уровень детализации, который позволяет ее понять.

Большая система никогда не является одним объектом

Когда начинающий разработчик впервые видит крупный проект, он часто говорит:

«Нужно разобраться в системе.»

Архитектор формулирует мысль иначе.

«Нужно понять, из каких частей состоит система.»

Это очень важное отличие.

Большая система почти никогда не является единым организмом.

Она представляет собой множество относительно самостоятельных частей, каждая из которых живет по собственным правилам.

Одни части тесно взаимодействуют между собой.

Другие почти не зависят друг от друга.

Некоторые изменяются ежедневно.

Другие остаются неизменными годами.

Архитектор начинает искать эти естественные различия.

Деление рождается не из технологий

Очень часто архитектуру начинают делить слишком поздно.

После выбора языка программирования.

После выбора базы данных.

После определения структуры каталогов.

После создания репозиториев.

Архитектор действует наоборот.

Он начинает делить систему еще до появления технологий.

Потому что причины разделения находятся не в программном обеспечении.

Они находятся в самом бизнесе.

Именно бизнес определяет, какие процессы связаны между собой, а какие существуют независимо.

Технологии лишь отражают эту структуру.

Они никогда не должны создавать ее.

Закон «Разделяй и понимай»

В политической литературе часто встречается принцип «Разделяй и властвуй».

Однако архитектор руководствуется другим принципом.

Разделяй, чтобы понимать.

Он не делит систему ради удобства разработки.

Не делит ее ради распределения задач между командами.

Не делит ради микросервисов.

Первое разделение преследует единственную цель.

Сделать систему понятной.

Только после этого можно говорить о программной архитектуре.

Первое разделение никогда не бывает окончательным

Когда архитектор впервые начинает выделять части системы, он прекрасно понимает, что эта модель еще будет меняться.

Некоторые границы окажутся неверными.

Некоторые процессы окажутся теснее связаны, чем казалось сначала.

Некоторые подсистемы придется объединить.

Другие — наоборот, разделить.

Поэтому первое разбиение нельзя воспринимать как окончательное архитектурное решение.

Это рабочая гипотеза.

Она помогает продолжить исследование.

Именно поэтому опытный архитектор не боится пересматривать собственную модель.

Он понимает, что уточнение модели — естественная часть проектирования.

Архитектор ищет естественные части

Самый важный вопрос этой главы звучит очень просто.

Что делает одну часть системы самостоятельной?

Ответ редко связан с программированием.

Обычно архитектор обращает внимание совсем на другие признаки.

Есть ли у этой части собственная цель?

Живет ли она по своим бизнес-правилам?

Может ли она изменяться независимо от остальных?

Использует ли она собственную терминологию?

Принимаются ли внутри нее собственные решения?

Если ответы на большинство этих вопросов положительные, перед архитектором, скорее всего, находится самостоятельная часть предметной области.

Именно такие части впоследствии становятся основой архитектуры.

Архитектор еще не знает названий

На этом этапе архитектор еще не думает о подсистемах.

Не говорит о модулях.

Не использует термин «контекст».

Не рисует границы микросервисов.

Все эти понятия появятся значительно позже.

Пока происходит другое.

Он постепенно начинает видеть, что сложная система состоит не из бесконечного множества элементов, а из нескольких относительно самостоятельных миров.

Это открытие становится одним из самых важных моментов архитектурного мышления.

Вместо заключения

Невозможно понять сложную систему, рассматривая ее как единое целое.

Человеческое мышление устроено иначе.

Оно стремится найти структуру.

Выделить закономерности.

Объединить связанные элементы.

Отделить независимые процессы.

Именно поэтому архитектор начинает проектирование не с компонентов, технологий или диаграмм.

Он начинает с поиска естественного устройства самой системы.

Потому что архитектура никогда не создается из случайного набора частей.

Она рождается тогда, когда найден порядок, который уже существовал внутри бизнеса.

Потому что тогда мы показываем настоящее архитектурное мышление: архитектор сначала упорядочивает собственное понимание реальности, и лишь затем принимает решения о структуре будущей информационной системы. Именно этот путь — от хаоса восприятия к структурированной модели — и составляет основу его профессиональной работы.

Глава 13. Где проходят настоящие границы системы

Архитектор ищет не модули, а границы

После того как архитектор понял, что сложная система состоит из нескольких самостоятельных частей, возникает следующий вопрос.

Где заканчивается одна часть системы и начинается другая?

На первый взгляд кажется, что ответ очевиден.

Можно разделить систему по меню.

По разделам пользовательского интерфейса.

По таблицам базы данных.

По командам разработки.

По микро сервисам.

Однако опыт показывает, что подобные границы редко совпадают с реальной структурой бизнеса.

Архитектор начинает искать совершенно другие признаки.

Его интересует не устройство программы.

Его интересует устройство предметной области.

Пользовательский интерфейс не определяет архитектуру

Очень часто систему начинают воспринимать через экраны.

Каталог.

Корзина.

Профиль пользователя.

Административная панель.

Личный кабинет.

Но пользовательский интерфейс создается для удобства человека.

Он показывает систему такой, какой ее удобно использовать.

Это вовсе не означает, что именно так она должна быть устроена внутри.

Один экран может одновременно работать с несколькими бизнес-процессами.

И наоборот.

Один бизнес-процесс может проходить через десятки различных экранов.

Интерфейс показывает способ взаимодействия пользователя с системой.

Архитектура описывает устройство самой системы.

Это разные задачи.

База данных тоже не определяет архитектуру

Еще одна распространенная ошибка заключается в том, что архитектуру начинают строить вокруг структуры базы данных.

Появляется несколько десятков таблиц.

После этого принимается решение:

каждая таблица станет отдельным модулем.

Или наоборот.

Все таблицы, связанные внешними ключами, объединяются в одну подсистему.

Однако база данных отражает способ хранения информации.

Она не объясняет, почему эта информация появилась.

Не описывает бизнес-правила.

Не показывает причины возникновения процессов.

Две сущности могут храниться рядом в одной таблице и при этом относиться к совершенно разным частям бизнеса.

И наоборот.

Один бизнес-процесс может использовать данные, находящиеся в десятках различных таблиц.

Поэтому архитектор никогда не ищет архитектуру внутри схемы базы данных.

Организационная структура компании тоже не является архитектурой

Иногда систему предлагают разделить по отделам компании.

Продажи.

Закупки.

Финансы.

Логистика.

Служба поддержки.

На первый взгляд такое разделение выглядит естественным.

Но организационная структура постоянно меняется.

Компании объединяют подразделения.

Создают новые департаменты.

Передают процессы на аутсорсинг.

Меняют зоны ответственности.

Если архитектура напрямую повторяет организационную структуру, то вместе с каждым изменением бизнеса приходится перестраивать всю систему.

Архитектор понимает, что организационная схема описывает ответственность людей.

Но не всегда отражает внутреннее устройство предметной области.

Команды разработки не должны определять границы системы

Иногда можно услышать:

«Этим сервисом занимается первая команда, а этим — вторая.»

Постепенно начинает казаться, что именно так и должна быть разделена система.

Но последовательность здесь обратная.

Сначала появляется архитектура.

Потом под нее формируются команды.

Если сделать наоборот, архитектура начнет зависеть от текущего состава разработчиков.

Через год команда изменится.

Через два года проект перейдет другой компании.

Архитектура останется.

Поэтому архитектор не определяет архитектуру только по текущему распределению разработчиков.

Настоящие границы рождаются внутри бизнеса

Возникает закономерный вопрос.

Если не пользовательский интерфейс.

Не база данных.

Не организационная структура.

Не команды разработки.

Тогда что определяет границы?

Ответ оказывается неожиданно простым.

Их определяет сам бизнес.

Архитектор начинает внимательно наблюдать за тем, как принимаются решения.

Какие процессы зависят друг от друга.

Какие правила всегда изменяются одновременно.

Какие понятия существуют независимо.

Какие события запускают другие события.

Постепенно начинают проявляться естественные области, внутри которых все тесно связано между собой.

Между такими областями взаимодействие существует.

Но оно значительно слабее.

Именно здесь проходят настоящие архитектурные границы.

Хорошая граница почти незаметна

Правильно найденная граница редко выглядит искусственной.

Наоборот.

После ее обнаружения возникает ощущение, что она существовала всегда.

Архитектор ничего не придумал.

Он лишь увидел закономерность, которая уже присутствовала в бизнесе.

Это похоже на работу геолога.

Геолог не создает тектонические плиты.

Он обнаруживает их.

Архитектор действует точно так же.

Он не рисует границы.

Он находит их.

Ошибочная граница всегда дорого обходится

Самая опасная ошибка архитектора — провести границу там, где ее не существует.

Последствия проявляются не сразу.

Появляются постоянные зависимости между подсистемами.

Изменение одной функции требует изменений в нескольких местах.

Одни и те же бизнес-правила начинают дублироваться.

Возникают циклические зависимости.

Команды постоянно мешают друг другу.

Со временем система становится все менее понятной.

Именно поэтому поиск правильных границ является одной из самых ответственных задач архитектора.

Исправить неверно проведенную границу спустя несколько лет значительно сложнее, чем потратить дополнительное время на ее поиск в самом начале проекта.

Архитектор не ищет идеальную границу

Важно понимать, что идеальных границ не существует.

Любая сложная система развивается.

Меняются процессы.

Появляются новые направления бизнеса.

Объединяются ранее независимые функции.

Разделяются старые процессы.

Поэтому архитектор ищет не идеальную границу.

Он ищет границу, которая наиболее точно отражает устройство бизнеса на текущем этапе его развития.

Архитектура должна быть достаточно устойчивой, чтобы выдерживать изменения, и достаточно гибкой, чтобы развиваться вместе с бизнесом.

Вместо заключения

Границы системы не определяются структурой каталогов проекта.

Не зависят от базы данных.

Не повторяют организационную структуру компании.

И не возникают потому, что так удобнее распределить работу между командами.

Настоящие границы появляются значительно раньше.

Они существуют внутри самой предметной области.

Архитектор не придумывает их.

Он внимательно исследует бизнес до тех пор, пока они не становятся очевидными.

Именно с этого момента начинается проектирование не программного обеспечения, а архитектуры системы.

Мне кажется, именно в этой главе можно сформулировать еще одну фундаментальную мысль, которая станет одной из опор всей книги.

Архитектура не должна отражать устройство программы. Она должна отражать устройство бизнеса.

На первый взгляд разница кажется незначительной. Но именно она объясняет, почему хорошие архитектуры живут десятилетиями, переживая смену технологий, языков программирования, фреймворков и команд разработки. Программа может быть переписана. База данных — заменена. Пользовательский интерфейс — полностью обновлен. Но если архитектура построена вокруг устойчивых закономерностей бизнеса, она сохранит свою ценность независимо от технологических изменений. Мне кажется, эта мысль станет прекрасным переходом к следующей главе, где мы начнем говорить о языке предметной области и о том, почему одинаковые слова в разных частях бизнеса могут означать совершенно разные вещи.

Глава 14. Почему одинаковые слова могут означать разные вещи

Архитектор начинает слушать слова

До этого момента архитектор изучал процессы.

Наблюдал за тем, как работает бизнес.

Искал естественные границы системы.

Но постепенно он начинает замечать еще одну закономерность.

Во время встреч все участники используют одни и те же слова.

Кажется, что все прекрасно понимают друг друга.

Однако архитектор не спешит делать такой вывод.

Он знает, что одинаковые слова далеко не всегда означают одно и то же.

Поэтому он начинает внимательно слушать не только ответы.

Он начинает слушать сами термины.

Одно слово — несколько смыслов

Представим обычное совещание.

Менеджер говорит:

— Нужно быстрее работать с клиентами.

Бухгалтер отвечает:

— Информация о клиентах должна храниться семь лет.

Сотрудник службы поддержки добавляет:

— История общения с клиентом должна быть доступна оператору.

Логист замечает:

— Клиент должен подтвердить получение груза.

Кажется, что разговор идет об одном и том же.

Но архитектор понимает другое.

Каждый из участников говорит о своем.

Для отдела продаж клиент — это потенциальный покупатель.

Для бухгалтерии — сторона договора.

Для службы поддержки — человек, который обращается за помощью.

Для логистики — получатель груза.

Используется одно слово.

Но за ним скрываются разные понятия.

Проблема не в словах

На первый взгляд может показаться, что достаточно договориться о правильной терминологии.

Но проблема значительно глубже.

Люди используют язык своей профессиональной области.

Именно этот язык помогает им эффективно работать.

Бухгалтер не обязан мыслить категориями логистики.

Логист не обязан использовать терминологию отдела продаж.

Специалисты по технической поддержке решают совершенно другие задачи.

Каждый из них прав.

Ошибка возникает тогда, когда архитектура начинает считать, что одинаковое слово всегда обозначает одну и ту же сущность.

Архитектор ищет не слова, а смыслы

Поэтому опытный архитектор почти никогда не спрашивает:

— Как это называется?

Вместо этого он задает совершенно другой вопрос:

— Что вы имеете в виду?

На первый взгляд разница кажется незначительной.

Но именно она позволяет избежать множества архитектурных ошибок.

Архитектор понимает, что слово является лишь обозначением.

Настоящий интерес представляет смысл, который вкладывает в него человек.

Именно смысл определяет бизнес-правила.

Именно смысл определяет поведение системы.

Именно смысл определяет место понятия в предметной области.

Иногда разные слова означают одно и то же

Возникает и обратная ситуация.

В разных подразделениях используют разные термины.

«Покупатель».

«Заказчик».

«Контрагент».

«Партнер».

«Получатель».

После анализа оказывается, что речь идет об одной и той же роли, рассматриваемой с разных сторон.

Архитектор не стремится немедленно заменить все слова одним термином.

Сначала он выясняет, действительно ли различия существуют только в языке.

Или за ними скрываются разные процессы и разные правила.

Слова меняются вместе с границами бизнеса

В предыдущей главе мы говорили о том, что архитектор ищет естественные границы системы.

Теперь становится понятно, почему это так важно.

Внутри каждой области деятельности постепенно формируется собственный язык.

Финансисты начинают использовать одни понятия.

Логисты — другие.

Специалисты по продажам — третьи.

И чем сложнее организация, тем сильнее отличаются эти языки.

Архитектор начинает понимать, что изменение смысла слова часто означает переход из одной области бизнеса в другую.

Таким образом язык сам начинает указывать на существующие границы.

Общий язык не создается приказом

Очень распространенная ошибка заключается в попытке сразу составить словарь терминов.

Архитектор понимает, что это не работает.

Нельзя приказать людям использовать одинаковые слова.

Язык складывается постепенно.

Он рождается в обсуждениях.

Уточняется в спорах.

Изменяется вместе с пониманием предметной области.

Общий язык становится следствием совместной работы над моделью предметной области, а не ее отправной точкой.

Архитектор становится переводчиком

Одна из самых необычных ролей архитектора заключается в том, что он постоянно переводит с одного профессионального языка на другой.

Он разговаривает с бизнесом.

С аналитиками.

С юристами.

С разработчиками.

С тестировщиками.

Службой эксплуатации.

Каждый использует собственную терминологию.

Архитектор постепенно сопоставляет эти языки.

Он замечает, какие понятия совпадают.

Какие различаются.

Какие противоречат друг другу.

Именно благодаря этому в проекте начинает формироваться единое понимание системы.

Язык становится частью архитектуры

Многие считают язык всего лишь способом общения.

Архитектор видит в нем значительно больше.

Язык определяет модель мышления.

Если участники проекта по-разному понимают одни и те же понятия, они неизбежно будут принимать разные архитектурные решения.

Поэтому работа с терминологией становится не редакторской задачей.

Она становится частью проектирования системы.

Чем точнее определены ключевые понятия, тем устойчивее оказывается архитектурная модель программной системы.

Вместо заключения

Архитектор внимательно относится к словам не потому, что стремится к идеальной терминологии.

Его задача значительно глубже.

Он пытается понять, какой смысл скрывается за каждым понятием.

Именно смысл определяет правила бизнеса.

Именно правила формируют модель предметной области.

А модель становится основой будущей архитектуры.

Поэтому для архитектора язык — это не средство общения.

Это один из главных инструментов исследования системы.

Архитектор понимает, что в большой системе не существует единственного правильного словаря. Каждая область бизнеса естественным образом формирует собственный язык, отражающий ее цели, процессы и правила. Поэтому задача архитектора состоит не в том, чтобы заставить всех говорить одинаково, а в том, чтобы понять, где различие в словах означает различие в смыслах.

Глава 15. Как архитектор начинает видеть независимые миры

Мир никогда не бывает единым

Когда архитектор только начинает изучать предметную область, ему кажется, что перед ним находится одна большая система.

Все процессы взаимосвязаны.

Все участники работают в одной компании.

Все используют одну информационную систему.

Все направлено на достижение общей цели.

Однако постепенно возникает странное ощущение.

Одни процессы практически не зависят от других.

Они живут собственной жизнью.

Подчиняются собственным правилам.

Используют собственный язык.

И принимают собственные решения.

Архитектор начинает понимать, что перед ним не один большой мир.

Перед ним несколько самостоятельных миров, которые лишь взаимодействуют друг с другом.

Каждый мир живет по своим законам

Представим крупную торговую компанию.

В ней существует отдел продаж.

Отдел логистики.

Финансовая служба.

Склад.

Юридический отдел.

Все они работают ради одной цели — успешной деятельности компании.

Но каждый из них живет по совершенно разным правилам.

Продажи думают о клиентах, коммерческих предложениях, переговорах и заключении сделок.

Логистика — о маршрутах, складах, перевозках и сроках доставки.

Финансы — о платежах, налогах, бюджетах и финансовой отчетности.

Юристы — о договорах, ответственности сторон и соблюдении законодательства.

Каждая из этих областей обладает собственной внутренней логикой.

Изменения происходят внутри своего мира

Архитектор замечает еще одну закономерность.

Большинство изменений происходит внутри одной области деятельности.

Изменились правила расчета скидок.

Это почти не затронуло логистику.

Появился новый способ доставки.

Продажи продолжают работать практически без изменений.

Изменилось налоговое законодательство.

Финансовая система перерабатывается.

Склад продолжает выполнять свои задачи.

Это означает, что изменения естественным образом группируются.

Они редко затрагивают всю систему одновременно.

Гораздо чаще они происходят внутри одного самостоятельного мира.

Именно это наблюдение становится одним из важнейших для архитектора.

Миры взаимодействуют, но не становятся одним целым

Полностью независимых областей почти не существует.

Продажа оборудования невозможна без логистики.

Логистика невозможна без информации о заказе.

Финансы должны знать о совершенной сделке.

Сервисное обслуживание зависит от поставленного оборудования.

Миры постоянно взаимодействуют.

Но очень важно понимать разницу между взаимодействием и смешением.

Продажи не должны жить по законам бухгалтерии.

Бухгалтерия не должна принимать решения за логистику.

Логистика не должна определять правила ремонта оборудования.

Каждая область отвечает за собственную часть реальности.

Она может обмениваться информацией с другими областями, но не должна перенимать их внутренние правила.

Архитектор начинает замечать границы ответственности

Постепенно архитектор перестает задавать вопрос:

«Какие функции должна выполнять эта подсистема?»

Вместо этого появляется другой вопрос:

«За какую часть реальности она отвечает?»

Это изменение кажется незначительным.

Но именно оно меняет весь подход к проектированию.

Подсистема перестает восприниматься как набор функций.

Она становится ответственным участником общей модели предметной области.

Каждая область отвечает за собственные решения.

Собственные правила.

Собственные данные.

Собственное развитие.

Именно ответственность становится главным признаком самостоятельного мира.

Независимость не означает изоляцию

Очень важно не сделать ошибочный вывод.

Если миры самостоятельны, это не означает, что между ними должны отсутствовать связи.

Наоборот.

Они постоянно взаимодействуют.

Передают информацию.

Реагируют на события.

Оказывают влияние друг на друга.

Но они не вмешиваются во внутреннюю жизнь соседних областей.

Хорошей аналогией может служить государство.

Министерство финансов взаимодействует с министерством транспорта.

Министерство транспорта взаимодействует с министерством здравоохранения.

Но каждое из них самостоятельно принимает решения в пределах своей компетенции.

Именно так архитектор начинает видеть устройство большой системы.

Архитектор перестает искать технические границы

На этом этапе становится очевидно, что речь идет вовсе не о программном обеспечении.

Архитектор еще не думает о микросервисах.

Не выбирает структуру каталогов.

Не определяет количество баз данных.

Он продолжает изучать сам бизнес.

И постепенно начинает видеть области, каждая из которых представляет собой отдельный мир со своей внутренней логикой.

Технологические решения появятся значительно позже.

Они должны будут лишь отразить уже найденную структуру.

Только потом появляются названия

Лишь спустя некоторое время архитектор обнаруживает, что подобный способ мышления давно существует в профессиональной литературе.

Разные авторы называют такие самостоятельные области по-разному.

Но суть остается неизменной.

Большая система состоит не из случайного набора компонентов.

Она состоит из относительно независимых миров, каждый из которых имеет собственную модель, собственный язык и собственные правила.

Позже архитектор узнает, что в предметно-ориентированном проектировании (Domain-Driven Design, DDD) такие миры получили название ограниченных контекстов (Bounded Contexts).

Однако к этому моменту термин уже ничего не объясняет.

Потому что понимание пришло раньше, чем название.

Вместо заключения

Архитектор начинает видеть систему совершенно иначе.

Она больше не выглядит монолитной.

Перед ним возникают самостоятельные области деятельности.

Каждая из них обладает собственной логикой.

Собственными правилами.

Собственным языком.

Собственными причинами изменений.

Эти области постоянно взаимодействуют между собой.

Но не растворяются друг в друге.

Именно поэтому хорошая архитектура строится не вокруг технологий, а вокруг естественных границ реального бизнеса.

Мы очень часто говорим, что архитектор делит систему на подсистемы. Но после предыдущих глав становится понятно, что это лишь внешнее проявление более глубокого процесса.

Архитектор не делит систему на части.

Он постепенно начинает видеть существующие миры, которые всегда были внутри бизнеса.

Это похоже на работу астронома. Ночью небо кажется россыпью случайных звезд. Но по мере накопления знаний становятся различимы созвездия, галактики, звездные скопления. Астроном не создает их — он учится их видеть.

Точно так же и архитектор. Сначала он видит хаос процессов, терминов и требований. Затем постепенно начинает различать самостоятельные миры предметной области. И только после этого становится возможным настоящее проектирование архитектуры.

Глава 16. Почему связи важнее компонентов

Архитектор сначала видит отношения

После того как архитектор начинает различать самостоятельные миры внутри системы, возникает следующий вопрос.

Каким образом эти миры существуют вместе?

На первый взгляд ответ кажется очевидным.

Они взаимодействуют.

Но именно здесь начинается настоящая архитектура.

Потому что архитектора интересует уже не существование отдельных частей.

Его интересует характер их взаимоотношений.

Именно отношения между частями превращают набор независимых областей в единую систему.

Компоненты существуют сами по себе

Представьте себе схему будущей системы.

На ней изображены десять прямоугольников.

Продажи.

Логистика.

Финансы.

Склад.

Инспекция.

Ремонт.

Каталог.

Управление пользователями.

Уведомления.

Отчетность.

Можно долго обсуждать каждый из этих компонентов.

Но пока между ними нет связей, невозможно понять, как работает система.

Перед нами всего лишь перечень областей деятельности.

Не более.

Архитектура начинается со стрелок

Теперь представим другую схему.

Между прямоугольниками появились линии.

Продажи передают заказ в логистику.

Логистика сообщает о доставке финансам.

Финансы подтверждают оплату.

Склад резервирует товар.

Инспекция публикует результаты проверки.

В этот момент схема начинает оживать.

Появляется движение.

Появляются причины и следствия.

Появляются зависимости.

Именно здесь возникает архитектура.

Не в квадратах.

А в связях между ними.

Каждая связь имеет цену

Очень легко провести линию на диаграмме.

Гораздо сложнее понять последствия.

Каждая новая зависимость означает, что одна часть системы начинает учитывать существование другой.

Она ожидает определенные данные.

Определенное поведение.

Определенные правила.

Чем больше таких зависимостей, тем сложнее становится изменять систему.

Одно изменение начинает распространяться по нескольким областям.

Со временем количество связей начинает расти быстрее, чем количество компонентов.

Именно поэтому архитектор относится к каждой новой зависимости крайне осторожно.

Хорошая архитектура — это не отсутствие связей

Иногда можно услышать мнение, что идеальная архитектура вообще не содержит зависимостей.

Это невозможно.

Любая реальная система представляет собой единое целое.

Ее части обязаны взаимодействовать.

Вопрос состоит совсем не в количестве связей.

Вопрос в их качестве.

Хорошая связь делает взаимодействие понятным.

Плохая связь создает скрытую зависимость.

Архитектор стремится не избавиться от связей.

Он стремится сделать их осмысленными.

Архитектор изучает направление зависимости

Для архитектора недостаточно знать, что две области взаимодействуют.

Ему необходимо понять характер этого взаимодействия.

Кто инициирует изменение?

Кто принимает решение?

Кто владеет информацией?

Кто зависит от результата?

Направление зависимости часто оказывается важнее самого факта взаимодействия.

Оно показывает, где находится источник истины.

Где принимаются решения.

И где проходят настоящие границы ответственности.

Не всякая связь должна быть прямой

Когда система только проектируется, возникает естественное желание соединить между собой все области, которым требуется обмен информацией.

Однако архитектор понимает, что большое количество прямых связей делает систему хрупкой.

Каждое изменение начинает затрагивать множество участников.

Поэтому он ищет способы сделать взаимодействие менее зависимым.

Иногда достаточно передать событие.

Иногда — опубликовать информацию.

Иногда — использовать посредника.

Способов существует много.

Но сама мысль остается неизменной.

Связь должна обеспечивать взаимодействие, а не создавать излишнюю зависимость.

Самые опасные зависимости невидимы

Не все связи изображаются на диаграммах.

Некоторые возникают незаметно.

Одна подсистема начинает использовать внутреннюю таблицу другой.

Модуль напрямую обращается к данным соседней области.

Разработчики копируют бизнес-правила из другого компонента.

Появляются общие библиотеки, которые постепенно начинают знать обо всех.

Такие зависимости редко обсуждаются.

Но именно они чаще всего становятся причиной деградации архитектуры.

Опытный архитектор постоянно ищет подобные скрытые связи.

Потому что именно они со временем начинают разрушать первоначальную модель системы.

Архитектор видит не линии, а последствия

Для человека, не занимающегося архитектурой, стрелка на диаграмме — это всего лишь линия.

Для архитектора каждая стрелка означает ответственность.

Изменение.

Риск.

Стоимость сопровождения.

Потенциальную точку отказа.

Дополнительное тестирование.

Дополнительные соглашения между командами.

Каждая новая зависимость увеличивает сложность системы.

Поэтому архитектор задает себе вопрос не «Можно ли провести эту связь?», а «Можно ли без нее обойтись?»

Простая диаграмма может быть сложнее большой

Иногда диаграмма содержит всего пять компонентов.

Но между ними проведено несколько десятков линий.

Такая архитектура оказывается значительно сложнее системы из двадцати компонентов, где каждый знает только то, что действительно необходимо.

Количество компонентов редко определяет сложность системы.

Настоящую сложность определяет сеть зависимостей.

Именно она показывает, насколько свободно система сможет развиваться в будущем.

Вместо заключения

Архитектор никогда не рассматривает компоненты отдельно друг от друга.

Для него они являются лишь носителями определенной ответственности.

Настоящий интерес представляют отношения между ними.

Именно связи показывают, как распространяются изменения.

Как принимаются решения.

Как движется информация.

Как взаимодействуют различные части бизнеса.

Поэтому хорошая архитектура определяется не количеством компонентов.

Она определяется качеством связей между ними.

Компоненты описывают состав системы. Связи описывают ее поведение.

Если убрать все компоненты, система исчезнет.

Но если оставить компоненты и убрать связи, система тоже перестанет существовать как система. Останется лишь набор изолированных частей.

Поэтому архитектор, рассматривая любую диаграмму, задает вопрос не «из чего состоит система?», а «как взаимодействуют ее части и почему именно так?»

Именно ответ на этот вопрос отличает архитектурную модель программной системы от набора прямоугольников.

Мысли вслух

:“Если посмотреть назад, то можно заметить, что предыдущие главы не учили читателя проектировать систему.

Они учили его видеть.

Сначала он научился понимать требования.

Потом — видеть предметную область.

Потом — находить границы.

Потом — замечать самостоятельные миры.

Потом — понимать язык бизнеса.

Потом — анализировать связи.

И теперь происходит очень интересный момент.

Все эти отдельные наблюдения неожиданно складываются в единую картину.

Именно так рождается архитектурная модель программной системы.

Не потому, что архитектор сел ее рисовать.

А потому, что она постепенно сформировалась в его понимании.”

Глава 17. Как рождается архитектурная модель программной системы

Архитектурная модель программной системы не появляется за один день

Со стороны может показаться, что однажды архитектор садится за доску, открывает редактор диаграмм и начинает проектировать систему.

На самом деле все происходит совершенно иначе.

К тому моменту, когда появляется первая схема, основная работа уже выполнена.

Архитектор уже изучил бизнес.

Понял требования.

Разобрался в противоречиях.

Выделил естественные границы.

Увидел самостоятельные области деятельности.

Понял, как они взаимодействуют между собой.

Диаграмма становится лишь отражением уже сложившегося понимания.

Архитектурная модель программной системы рождается значительно раньше.

Модель возникает постепенно

Нельзя назвать момент, когда модель появляется окончательно.

Она складывается постепенно.

Каждая встреча с заказчиком уточняет отдельные детали.

Каждый новый вопрос позволяет увидеть очередную закономерность.

Каждое найденное противоречие заставляет пересмотреть прежние представления.

Каждое новое бизнес-правило уточняет устройство предметной области.

Сначала архитектор видит отдельные элементы.

Потом связи между ними.

Затем начинает понимать причины этих связей.

И однажды приходит ощущение, что вся система начинает восприниматься как единое целое.

Именно в этот момент рождается архитектурная модель программной системы.

Архитектор моделирует не программу

Это одна из самых распространенных ошибок.

Очень часто архитектурную модель программной системы начинают воспринимать как схему будущего программного обеспечения.

Однако на данном этапе никакой программы еще не существует.

Архитектор моделирует сам бизнес.

Он отвечает совсем на другие вопросы.

Какие процессы существуют?

Какие решения принимаются?

Кто отвечает за эти решения?

Какие области независимы?

Какие взаимодействуют между собой?

Какие изменения происходят совместно?

Пока речь идет не о коде.

Речь идет об устройстве реального мира, который впоследствии должна отразить информационная система.

В модели нет технологий

На этом этапе архитектор еще не обсуждает языки программирования.

Не выбирает базы данных.

Не определяет количество сервисов.

Не принимает решение о монолите или микросервисах.

Не обсуждает способы интеграции.

Не проектирует программные интерфейсы.

Все эти вопросы преждевременны.

Потому что пока еще неизвестно самое главное.

Как устроен сам бизнес.

Если модель предметной области построена неверно, никакая технология не сможет исправить эту ошибку.

Хорошая модель переживает технологии

История разработки программного обеспечения постоянно меняется.

Появляются новые языки программирования.

Меняются базы данных.

Возникают новые способы взаимодействия между системами.

Одни архитектурные стили становятся популярными.

Другие постепенно уходят.

Но правильно построенная модель предметной области остается удивительно устойчивой.

Она отражает не состояние технологий.

Она отражает устройство бизнеса.

Именно поэтому хорошие архитектуры способны пережить несколько поколений программного обеспечения.

Модель помогает принимать решения

Архитектурная модель программной системы нужна вовсе не для красивых диаграмм.

Она становится инструментом принятия решений.

Когда появляется новое требование, архитектор задает вопрос:

Где ему место в существующей модели?

Если появляется новый процесс:

Какой мир бизнеса за него отвечает?

Если возникает новая интеграция:

Какие области она должна связать?

Если меняются бизнес-правила:

Какая часть модели должна измениться?

Модель превращается в своеобразную карту предметной области.

Она помогает ориентироваться в системе независимо от ее размеров.

Хорошая модель делает систему предсказуемой

Опытный архитектор постепенно начинает замечать интересную особенность.

Если модель построена правильно, многие решения становятся очевидными.

Новый процесс сам находит свое место.

Новая функция естественным образом относится к определенной области.

Новая интеграция становится логическим продолжением существующих связей.

Архитектор перестает принимать решения интуитивно.

Модель начинает подсказывать правильное направление.

Архитектурная модель программной системы никогда не бывает законченной

Бизнес развивается.

Появляются новые направления.

Исчезают старые процессы.

Меняются требования рынка.

Изменяется законодательство.

Расширяется предметная область.

Вместе с этим развивается и архитектурная модель программной системы.

Она не является документом, который однажды утверждается и остается неизменным.

Она представляет собой постоянно уточняемое описание бизнеса.

Поэтому архитектор никогда не говорит:

«Модель завершена.»

Он говорит:

«На сегодняшний день она наиболее полно отражает устройство предметной области.»

Архитектор начинает думать моделями

На определенном этапе происходит почти незаметное изменение профессионального мышления.

Архитектор перестает думать отдельными функциями.

Перестает воспринимать систему как набор экранов или сервисов.

Перед ним возникает целостная модель.

Любое новое требование он сначала помещает внутрь этой модели.

И только потом начинает обсуждать способы его реализации.

Именно этот момент отличает архитектора от разработчика.

Разработчик чаще думает реализацией.

Архитектор — моделью.

Вместо заключения

Архитектурная модель программной системы не является диаграммой.

Не является документом.

Не является описанием программного обеспечения.

Она представляет собой мысленную конструкцию, которая объясняет, как устроен бизнес и почему его части взаимодействуют именно так.

Все последующие архитектурные решения будут опираться именно на нее.

Выбор технологий.

Разделение системы.

Проектирование сервисов.

Построение интерфейсов.

Организация хранения данных.

Интеграция с внешними системами.

Все это станет следствием модели, а не ее причиной.

Поэтому архитектор сначала строит модель предметной области.

И только затем начинает строить информационную систему.

Архитектурная модель программной системы — это не проект будущей системы. Это объяснение того, как устроен бизнес.

В этой фразе скрыто принципиальное различие между архитектурным мышлением и проектированием программного обеспечения.

Если архитектор начинает с технологий, он неизбежно подгоняет бизнес под выбранные инструменты.

Если же он начинает с модели предметной области, технологии становятся всего лишь способом ее реализации.

Именно поэтому в этой главе еще нет ни микросервисов, ни баз данных, ни очередей сообщений, ни API, ни облачных платформ. Они обязательно появятся в следующих частях книги, но к этому моменту читатель уже поймет главное: архитектура рождается не из технологий, а из понимания предметной области.

Глава 18. Когда архитектор впервые рисует схему

Большинство начинает со схемы

Практически любой архитектурный проект начинается одинаково.

Открывается редактор диаграмм.

На доске появляются прямоугольники.

Между ними рисуются стрелки.

Обсуждаются сервисы.

Базы данных.

Очереди сообщений.

Появляется ощущение, что архитектура постепенно рождается прямо на экране.

Но опыт показывает совершенно другую картину.

Очень часто такая схема не помогает понять систему.

Она лишь фиксирует непонимание.

Красиво оформленное.

Аккуратно подписанное.

Но все равно непонимание.

Архитектор долго не рисует ничего

Опытный архитектор может несколько дней или даже недель обходиться вообще без схем.

Со стороны это выглядит странно.

Кажется, что работа не движется.

Однако в действительности происходит самое важное.

Архитектор думает.

Слушает.

Спрашивает.

Сомневается.

Проверяет собственные выводы.

Возвращается к уже принятым решениям.

Постепенно в его сознании начинает складываться модель будущей системы.

Именно эта работа является настоящим проектированием.

Все остальное появится позже.

Модель рождается раньше схемы

К тому моменту, когда архитектор берет в руки маркер или открывает графический редактор, большая часть решений уже принята.

Он уже понимает:

где проходят границы;

какие существуют самостоятельные миры;

как они взаимодействуют;

где возникают зависимости;

какие изменения происходят совместно;

какие правила принадлежат каждой области.

Поэтому первая схема появляется не в начале работы.

Она появляется в тот момент, когда становится понятно, что именно необходимо показать.

Схема ничего не объясняет сама по себе

Очень часто можно услышать:

«Нарисуйте архитектуру, и все станет понятно.»

Это опасное заблуждение.

Любая схема является всего лишь графическим представлением определенной модели.

Если сама модель ошибочна, никакая схема не сможет сделать ее правильной.

Она лишь придаст ошибке наглядную форму.

Поэтому опытный архитектор никогда не доверяет красивой картинке.

Он всегда задает вопрос:

Какое понимание стоит за этой схемой?

Хорошая схема делает невидимое видимым

Архитектурная схема не должна показывать все.

Она должна показывать главное.

Она делает видимыми те закономерности, которые невозможно удерживать в памяти одновременно.

Она позволяет другим людям увидеть ту модель, которая уже сформировалась у архитектора.

Именно поэтому хорошая схема всегда проще самой системы.

Она убирает случайные детали.

Не отвлекает на второстепенные элементы.

Подчеркивает только те взаимосвязи, которые действительно важны для обсуждаемого вопроса.

Одна модель — множество схем

Еще одна ошибка заключается в попытке создать единственную «правильную» схему системы.

Архитектор знает, что такой схемы не существует.

Одна и та же модель может быть представлена совершенно по-разному.

Руководителю проекта важно увидеть потоки работ.

Разработчику — взаимодействие компонентов.

Аналитику — бизнес-процессы.

Специалисту по эксплуатации — развертывание системы.

Все они смотрят на одну и ту же архитектуру.

Но каждому требуется собственная схема.

Меняется способ представления.

Модель при этом остается неизменной.

Если схему невозможно объяснить словами — она еще не готова

Опытный архитектор редко начинает разговор с демонстрации схемы.

Сначала он объясняет идею.

Рассказывает о границах.

О независимых областях.

О причинах их появления.

О характере взаимодействия.

И только потом показывает схему.

Потому что схема должна подтверждать понимание, а не заменять его.

Если архитектуру невозможно последовательно объяснить без рисунка, скорее всего, она еще недостаточно осмыслена.

Схема — это средство общения

Архитектор рисует схемы не для себя.

Они нужны для совместной работы.

С их помощью обсуждаются идеи.

Проверяются гипотезы.

Выявляются ошибки.

Передаются знания.

Поэтому хорошая схема должна быть понятна не только своему автору.

Она должна помогать другим людям увидеть ту же модель, которую видит архитектор.

Если этого не происходит, значит схема не выполнила свою главную задачу.

Хорошая схема всегда несовершенна

Многие стремятся создать идеальную архитектурную схему.

Такую, в которой будет отражено абсолютно все.

Но именно такие схемы чаще всего становятся бесполезными.

Они перегружены деталями.

Сложны для восприятия.

Трудны в сопровождении.

Архитектор понимает, что схема всегда является упрощением реальности.

Именно поэтому он сознательно оставляет за ее пределами множество деталей.

Не потому, что они не важны.

А потому, что каждая схема должна отвечать только на один вопрос.

Как только она начинает отвечать на десять вопросов одновременно, она перестает отвечать хотя бы на один.

Вместо заключения

Архитектор не начинает проектирование со схемы.

Он приходит к ней.

Сначала рождается понимание.

Затем появляется модель.

И только после этого возникает необходимость зафиксировать ее в графическом виде.

Поэтому схема не является источником архитектуры.

Она становится способом передать архитектурное мышление другим людям.

Именно поэтому опытный архитектор может долго обходиться без единого рисунка.

Но когда первая схема появляется, она оказывается не началом работы, а ее естественным результатом.

Схема помогает другим людям понять архитектора. Архитектор уже все знает!

Опытный архитектор рисует схему тогда, когда уже разобрался.

Его задача — не найти решение на бумаге, а сделать свое понимание доступным для других участников проекта.

Сначала архитектор формирует модель в своем мышлении, и лишь затем превращает ее в схему, которая становится общим языком для всей команды.

Часть IV. Архитектура предметной области или как модель становится программой

Глава 19. Почему архитектура начинается не с программного обеспечения

Архитектура начинается гораздо раньше

Большинство людей связывает архитектуру с программным обеспечением.

Когда произносится слово «архитектор», почти сразу возникают привычные ассоциации.

Языки программирования.

Микросервисы.

Базы данных.

Очереди сообщений.

Облачные платформы.

Контейнеризация.

Кажется очевидным, что именно здесь начинается работа архитектора.

Но практика показывает совершенно другую картину.

К тому моменту, когда архитектор начинает обсуждать технологии, значительная часть архитектурных решений уже принята.

И принята она была задолго до появления первого программного компонента.

До появления программного обеспечения уже существует бизнес

Любая информационная система создается не сама по себе.

Она появляется потому, что существует определенная деятельность.

Компания продает товары.

Банк обслуживает клиентов.

Больница оказывает медицинскую помощь.

Логистическая компания организует перевозки.

Страховая компания оценивает риски.

Во всех этих случаях бизнес существовал раньше, чем появилась информационная система.

Существовали процессы.

Существовали правила.

Существовали роли.

Существовала ответственность.

Люди принимали решения.

Заключали договоры.

Передавали документы.

Несли обязательства.

Информационная система появляется значительно позже.

Она не создает этот мир.

Она лишь пытается его описать.

Архитектор изучает не программу

Именно поэтому опытный архитектор начинает свою работу совсем не с программного обеспечения.

Он изучает компанию.

Наблюдает за работой людей.

Разбирается в процессах.

Выясняет причины принятия решений.

Пытается понять, почему именно так устроен бизнес.

На этом этапе ему совершенно безразлично, какая база данных будет использоваться.

Будет ли система монолитной или распределенной.

Какой язык программирования выберет команда.

Все эти вопросы пока не имеют значения.

Потому что архитектор решает совсем другую задачу.

Он пытается понять устройство предметной области.

Сначала появляется понимание, потом появляется модель

Во время предыдущих частей книги мы уже прошли этот путь.

Мы анализировали требования.

Искали противоречия.

Выделяли естественные границы.

Размышляли о языке бизнеса.

Учились видеть самостоятельные области деятельности.

Исследовали связи между ними.

Постепенно у нас сформировалась архитектурная модель программной системы.

Заметим одну интересную особенность.

За все это время мы практически не говорили о программировании.

Мы не обсуждали базы данных.

Не выбирали архитектурные стили.

Не сравнивали технологии.

И тем не менее архитектура уже начала складываться.

Потому что архитектура начинается не с реализации.

Она начинается с понимания.

Программа отражает модель, а не наоборот

Очень распространенной ошибкой является попытка строить модель, исходя из структуры будущего программного обеспечения.

Появляются классы.

Появляются сервисы.

Появляются таблицы базы данных.

И постепенно возникает иллюзия, что именно они определяют устройство системы.

На самом деле происходит обратное.

Если архитектор правильно понял бизнес, структура программного обеспечения становится естественным продолжением этой модели.

Если же понимание оказалось ошибочным, никакая технология не сможет исправить фундаментальную проблему.

Можно изменить язык программирования.

Можно заменить систему хранения данных.

Можно переписать программный код.

Но невозможно компенсировать неправильное понимание предметной области.

Архитектор проектирует реальность

Это утверждение сначала кажется странным.

Архитектор не проектирует деятельность компании.

Она уже существует.

Но он проектирует способ ее отражения в информационной системе.

Для этого ему необходимо понять, что действительно важно.

Какие понятия являются основными.

Какие правила нельзя нарушать.

Какие процессы определяют работу организации.

Какие решения принимаются людьми.

Какие события меняют состояние бизнеса.

Именно эта модель становится основой будущей системы.

Не наоборот.

Почему технологии появляются позже

Многие начинающие архитекторы стремятся как можно раньше перейти к выбору технологий.

Им кажется, что именно это является главной частью профессии.

Но опыт показывает, что одна и та же модель предметной области может быть реализована совершенно разными средствами.

Сегодня это может быть монолит.

Завтра — модульная система.

Позже — распределенная архитектура.

Через несколько лет могут измениться языки программирования, базы данных, средства интеграции или платформы развертывания.

Но если модель предметной области построена правильно, все эти изменения останутся техническими.

Они не потребуют заново понимать сам бизнес.

Архитектура начинается с понимания мира

Постепенно становится очевидной одна закономерность.

Архитектор работает сразу с двумя мирами.

Первый мир существует независимо от программного обеспечения.

Это реальный бизнес.

Реальные процессы.

Реальные люди.

Реальная ответственность.

Второй мир появляется позже.

Это информационная система, которая должна максимально точно отразить устройство первого мира.

Поэтому архитектура начинается не в момент написания первого класса.

И даже не в момент выбора технологий.

Она начинается тогда, когда архитектор начинает понимать устройство реального мира.

У этой идеи уже есть имя

Все, о чем говорилось в предыдущих главах, может показаться исключительно авторским способом рассуждения.

Однако это не так.

На протяжении многих десятилетий инженеры, архитекторы и исследователи пытались ответить на один и тот же вопрос.

Как создавать информационные системы, которые отражают устройство сложного бизнеса, а не подменяют его особенностями выбранной технологии?

В результате постепенно сформировался подход, который получил название предметно-ориентированное проектирование (Domain-Driven Design, DDD).

Его целью является не изучение программного кода.

Не выбор технологий.

Не описание архитектурных шаблонов.

Его основная задача значительно глубже.

Он помогает архитектору понять предметную область и построить такую модель предметной области, которая сможет стать надежной основой будущей информационной системы.

Вместо заключения

Если после прочтения этой главы у читателя возникло ощущение:

«Получается, все предыдущие главы уже были разговором о предметно-ориентированном проектировании, просто без специальных терминов»,

значит, мы достигли своей цели.

Именно так и должно было произойти.

Сначала появилось понимание.

Потом — модель.

И только теперь пришло время познакомиться с тем языком, который инженерное сообщество выработало для описания этих идей.

Следующие главы будут посвящены не изучению новой философии.

Они будут посвящены тому, чтобы дать уже известным нам мыслям профессиональные имена и показать, как они применяются при проектировании сложных информационных систем.

Архитектор не начинает изучение предметно-ориентированного проектирования. Он обнаруживает, что давно мыслит теми же принципами, которые инженерное сообщество однажды сформулировало и объединило в единую систему знаний.

Я не учу читателя новой терминологии. Я показываю, что настоящие архитектурные принципы рождаются из наблюдения за реальным бизнесом, а профессиональные термины лишь помогают архитекторам говорить об этих принципах на одном языке. Именно поэтому читатель не будет запоминать определения — он будет узнавать в них собственный путь, который уже прошел вместе с книгой.

Глава 20. Где заканчивается предметная область

Архитектор не может изучить весь мир

Когда архитектор начинает новый проект, перед ним открывается огромная картина.

Компания.

Сотрудники.

Подразделения.

Партнеры.

Клиенты.

Документы.

Производство.

Продажи.

Финансы.

Закупки.

Логистика.

Юридическая служба.

Маркетинг.

Служба безопасности.

Кажется, что все это необходимо изучить.

И действительно, все эти области связаны между собой.

Но очень быстро становится понятно, что такой подход невозможен.

Если пытаться понять абсолютно все, проект никогда не начнется.

Любая система отвечает только за часть бизнеса

Представим крупную производственную компанию.

В ней одновременно работают десятки информационных систем.

Одна отвечает за бухгалтерский учет.

Другая — за управление персоналом.

Третья — за склад.

Четвертая — за техническое обслуживание оборудования.

Пятая — за работу с клиентами.

Все они существуют внутри одной организации.

Но каждая из них решает собственную задачу.

Ни одна не пытается описать весь бизнес целиком.

Именно поэтому каждая информационная система исследует лишь ту часть реального мира, ради которой она создается.

Самое трудное решение архитектора

На первый взгляд кажется, что архитектору нужно понять как можно больше.

На практике происходит обратное.

Одним из первых и самых важных решений становится определение того, что не относится к проектируемой системе.

Это решение требует профессиональной зрелости.

Если включить в модель слишком много, система станет громоздкой и потеряет четкие границы.

Если исключить действительно важные процессы, она перестанет отражать реальность.

Архитектор постоянно ищет баланс между полнотой и необходимостью.

Границы определяются целью системы

Представим компанию, которая занимается производством промышленного оборудования.

Если создается система управления ремонтами, архитектора интересуют:

- оборудование;
- техническое состояние;
- виды ремонтов;
- графики обслуживания;
- запасные части;
- история эксплуатации.

При этом бухгалтерский учет, расчет заработной платы или маркетинговые кампании могут существовать в этой же компании, но для данной системы они остаются за пределами исследования.

Не потому, что они не важны.

А потому, что они не помогают решать задачу, ради которой создается система.

Таким образом, границы определяются не структурой организации, а назначением самой информационной системы.

Предметная область всегда имеет границы

Именно здесь появляется одна из важнейших мыслей всей книги.

Предметная область никогда не совпадает со всей деятельностью компании.

Она всегда ограничена.

Это сознательно выбранная часть реального мира, которую архитектор исследует, моделирует и затем отражает в информационной системе.

Все, что остается за этими границами, продолжает существовать.

Иногда такие области взаимодействуют с нашей системой.

Иногда они оказывают на нее влияние.

Но сами по себе они уже не являются частью той модели, которую строит архитектор.

У этого понятия есть профессиональное название

В инженерной практике та часть реального мира, которая исследуется и моделируется при создании конкретной информационной системы, называется предметной областью (Domain).

Важно обратить внимание на одну деталь.

Речь идет не обо всем бизнесе компании.

И не обо всех процессах организации.

Domain — это только та область деятельности, ради понимания и поддержки которой создается данная система.

Именно поэтому в одной компании может существовать несколько различных предметных областей.

Каждая из них имеет собственные цели, собственные понятия и собственную модель.

Архитектор определяет не только то, что войдет в систему

Есть распространенное заблуждение, что архитектор определяет состав будущей системы.

На самом деле он делает нечто более важное.

Он определяет границы исследования.

Именно это решение влияет на все последующие этапы проектирования.

Какие понятия будут считаться основными.

Какие процессы окажутся значимыми.

Какие правила войдут в модель.

Какие события будут иметь значение.

И наоборот — какие части реального мира останутся за пределами архитектуры.

Без этих границ невозможно построить целостную модель.

Вместо заключения

Понять предметную область — значит не изучить весь бизнес.

Это значит определить, какая часть реального мира действительно относится к создаваемой системе.

Архитектор начинает не с поиска ответов.

Он начинает с определения границ вопросов.

И только после этого появляется возможность строить модель.

Потому что невозможно описать мир целиком.

Но можно глубоко понять ту его часть, за которую будет отвечать будущая информационная система.

Опытный архитектор отличается не тем, что изучает больше. Он отличается тем, что умеет вовремя остановиться.

Это очень зрелая мысль. Молодой специалист часто стремится охватить как можно больше информации, опасаясь что-то упустить. Архитектор же понимает: попытка смоделировать весь мир неизбежно приводит к размытой, перегруженной и противоречивой модели.

Поэтому одно из первых архитектурных решений — не выбор технологии и даже не построение модели. Это решение о том, где проходят границы исследования. Именно с этого момента начинается настоящая работа с предметной областью, и именно здесь термин Domain перестает быть определением из книги и превращается в практический инструмент архитектора

Глава 21. Как предметная область превращается в программную модель

Понимания бизнеса недостаточно

К этому моменту архитектор уже знает достаточно много.

Он изучил предметную область.

Понял устройство бизнеса.

Определил границы исследования.

Выделил основные понятия.

Разобрался в правилах и процессах.

Кажется, что теперь можно приступать к программированию.

Однако именно здесь начинается новая работа.

Понимание бизнеса еще не является программной системой.

Между ними существует этап, который определяет качество всей будущей архитектуры.

Архитектору предстоит превратить знания о предметной области в программную модель.

Модель не переносится в программу автоматически

Иногда кажется, что этот переход очевиден.

Если бизнес описан достаточно хорошо, остается лишь реализовать его в программном коде.

На практике все значительно сложнее.

Не каждое понятие предметной области должно стать программным объектом.

Не каждое бизнес-правило обязана контролировать информационная система.

Не каждый процесс имеет смысл автоматизировать.

Архитектор начинает задавать вопросы, которых раньше не существовало.

Какие понятия действительно должны появиться в программе?

Какие правила система обязана обеспечивать самостоятельно?

Какие действия останутся ответственностью человека?

Какие события необходимо сохранять?

Какие данные являются существенными, а какие можно не учитывать?

Именно ответы на эти вопросы постепенно превращают модель предметной области в модель программной системы.

Программная модель — это инженерное решение

Важно понимать, что программная модель не является копией бизнеса.

Она создается для решения конкретной задачи.

Поэтому архитектор сознательно упрощает реальность.

Он сохраняет только те элементы предметной области, без которых система не сможет выполнять свое назначение.

Все остальное остается за пределами модели.

Именно поэтому две информационные системы, работающие в одной компании, могут по-разному описывать один и тот же бизнес.

Каждая из них решает собственную задачу и строит собственную программную модель.

Архитектура меняет не только программу

На этом этапе происходит то, о чем редко говорят в книгах.

До сих пор казалось, что работа движется только в одном направлении.

Бизнес формулирует требования.

Архитектор их анализирует.

Программная система реализует полученную модель.

Но во время проектирования возникает и обратный процесс.

Инженерная реализация обнаруживает ограничения, которые невозможно увидеть при изучении бизнеса.

Например, компания принимает решение регистрировать пользователей по электронной почте, с помощью SMS или через мессенджер.

С точки зрения бизнеса этого достаточно.

Однако при проектировании системы появляются вопросы, которых раньше не существовало.

Сколько времени должен действовать код подтверждения?

Сколько раз пользователь может запросить новый код?

Как защититься от автоматических регистраций?

Как предотвратить подбор кодов подтверждения?

Как ограничить массовую отправку сообщений?

Как обнаружить подозрительную активность?

Ни один из этих вопросов не возникает в предметной области сам по себе.

Они появляются тогда, когда бизнес-модель начинает превращаться в программную систему.

И очень часто ответы на них изменяют первоначальные требования.

В результате бизнес начинает учитывать ограничения, продиктованные инженерной практикой.

Архитектор соединяет два мира

Именно в этом заключается одна из важнейших особенностей профессии архитектора.

Он не является переводчиком требований.

Он соединяет две разные реальности.

С одной стороны, архитектор переносит понимание бизнеса в программную модель.

С другой — возвращает в бизнес знания об инженерных ограничениях, требованиях безопасности, надежности, производительности и эксплуатации.

В результате меняется не только программа.

Меняется и сама организация работы.

Хорошая архитектура рождается не тогда, когда одна сторона диктует правила другой.

Она возникает тогда, когда бизнес и инженерная практика постепенно приходят к общему пониманию того, какой должна быть система.

Предметно-ориентированное проектирование как инструмент архитектора

Именно здесь предметно-ориентированное проектирование (Domain-Driven Design, DDD) перестает быть набором терминов.

Оно становится практическим инструментом.

Его задача состоит не в том, чтобы навязать определенный способ написания кода.

Его задача — помочь сохранить связь между предметной областью и программной системой на протяжении всего жизненного цикла проекта.

Чем точнее эта связь, тем устойчивее архитектура к изменениям бизнеса.

Вместо заключения

Переход от предметной области к программной модели — это не механический перенос требований в код.

Это совместная работа бизнеса и инженерии.

Бизнес определяет, что должно делать будущее решение.

Инженерная практика помогает понять, как это можно реализовать надежно, безопасно и эффективно.

Именно на этом этапе рождается архитектура программной системы.

Архитектор строит мост между двумя мирами. По одному направлению он переносит понимание бизнеса в программную модель. По другому — возвращает в бизнес инженерные знания, без которых современная информационная система не сможет существовать.

До этого читатель учился понимать бизнес. Теперь он понимает, что архитектура — это не односторонний процесс, а постоянный диалог между предметной областью и инженерной реальностью. Именно этот диалог и превращает знания о бизнесе в жизнеспособную программную систему.

Глава 22. Как архитектура начинает говорить на языке бизнеса

Хорошая архитектура узнаваема

После того как архитектор построил программную модель, возникает следующий вопрос.

Как организовать саму программу?

На первый взгляд ответ кажется простым.

Создать каталоги.

Разделить код на модули.

Определить зависимости.

Выбрать архитектурный стиль.

Однако опыт показывает, что именно на этом этапе многие проекты начинают постепенно терять связь с предметной областью.

Причина заключается не в ошибках программирования.

Она значительно глубже.

Архитектура перестает говорить на языке бизнеса.

Программа должна отражать предметную область

Если архитектор правильно выполнил предыдущую работу, то к этому моменту он уже понимает устройство бизнеса.

Теперь это понимание должно проявиться в самой структуре программной системы.

Разработчик, впервые открывший проект, должен увидеть не набор технических компонентов, а отражение предметной области.

Названия модулей, компонентов и основных объектов должны быть понятны не только программистам, но и людям, хорошо знающим бизнес.

Именно поэтому хорошие архитектуры редко начинаются с каталогов вроде:

- utils;
- common;
- helpers;
- misc;
- service1;
- service2.

Подобные названия ничего не говорят о назначении системы.

Они описывают лишь технический способ организации файлов.

Архитектор стремится к другому.

Структура программы должна отражать устройство предметной области.

Язык бизнеса становится языком программы

По мере развития проекта становится очевидно, что одних диаграмм недостаточно.

Если участники проекта используют разные названия для одних и тех же понятий, архитектура постепенно начинает разрушаться.

Поэтому архитектор стремится к тому, чтобы бизнес и разработчики говорили на одном языке.

В предметно-ориентированном проектировании (Domain-Driven Design, DDD) этот подход получил название единый язык (Ubiquitous Language).

Однако сама идея значительно старше любого термина.

Если все участники проекта одинаково понимают смысл основных понятий, вероятность архитектурных ошибок существенно уменьшается.

Поэтому хороший архитектор сначала договаривается о языке, а уже потом принимает технические решения.

Архитектура повторяет устройство бизнеса

По мере развития системы архитектор замечает еще одну закономерность.

Даже внутри одной предметной области существуют части, которые живут по собственным правилам.

Например, управление заказами, складской учет, логистика и расчеты могут использовать похожие понятия, но вкладывать в них различный смысл.

Попытка объединить их в единую программную модель обычно приводит к усложнению системы.

Поэтому архитектор определяет границы, внутри которых действует единая модель предметной области, а понятия и правила понимаются согласованно.

В предметно-ориентированном проектировании такую границу применимости модели называют ограниченным контекстом (Bounded Context).

Но для архитектора важен не сам термин.

Важно понять принцип.

Если бизнес разделен на самостоятельные области ответственности, то и программная система должна отражать это разделение.

Архитектура становится понятной без объяснений

Хорошую архитектуру отличает одно важное свойство.

Она не требует длительных объяснений.

Опытный разработчик способен открыть проект и уже по его структуре понять, чем занимается система.

Он видит названия модулей.

Понимает их ответственность.

Замечает границы между частями программы.

Представляет основные бизнес-процессы.

Для этого ему не нужно изучать документацию в течение нескольких недель.

Архитектура сама становится частью документации.

Она говорит с разработчиком на языке предметной области.

Архитектура — это язык

Со стороны может показаться, что архитектор проектирует программные компоненты.

На самом деле он проектирует язык, на котором программа будет описывать предметную область.

Именно этот язык объединяет бизнес, аналитиков, архитекторов и разработчиков.

Пока все участники проекта понимают его одинаково, система развивается естественно.

Когда общий язык теряется, архитектура постепенно перестает отражать бизнес и начинает жить собственной жизнью.

Именно с этого момента обычно начинаются проблемы, которые позже называют «техническим долгом».

Вместо заключения

Архитектура становится зрелой не тогда, когда в ней появляются сложные технические решения.

Она становится зрелой тогда, когда структура программы начинает естественно отражать устройство предметной области.

В этот момент программа перестает быть просто набором файлов и классов.

Она становится инженерной моделью предметной области, выраженной средствами программного обеспечения.

Если для понимания структуры программы необходимо сначала изучить техническую документацию, значит архитектура говорит на языке технологий. Если же структура проекта понятна

человеку, знающему предметную область, значит архитектура говорит на языке бизнеса.

Глава 23. Что архитектор действительно берет из DDD

К этому моменту многое уже стало знакомым

Если читатель внимательно прошел предыдущие главы, то мог заметить одну особенность.

Многие идеи, которые постепенно появлялись по ходу книги, давно известны профессиональному сообществу.

Мы говорили о необходимости понимать бизнес прежде, чем проектировать систему.

О том, что архитектура начинается с определения границ предметной области.

О том, что программа должна отражать устройство бизнеса.

О том, что единый язык снижает количество ошибок.

О том, что сложная система неизбежно разделяется на самостоятельные части.

Все эти идеи не появились случайно.

Они были сформулированы и объединены в рамках предметно-ориентированного проектирования (Domain-Driven Design, DDD).

Однако в нашей книге мы сознательно пришли к ним другим путем.

Сначала через мышление архитектора.

И только затем — через инженерную практику.

DDD не учит понимать бизнес

Иногда создается впечатление, что достаточно изучить DDD, чтобы научиться проектировать сложные системы.

На практике все происходит иначе.

DDD не заменяет понимание предметной области.

Он не проводит интервью с заказчиком.

Не выявляет противоречия в требованиях.

Не помогает договориться между подразделениями компании.

Не принимает архитектурные решения.

Все это остается задачей архитектора.

DDD начинает приносить пользу только тогда, когда архитектор уже понимает предметную область и ищет способ сохранить это понимание в программной системе.

DDD дает архитектору язык

Любая зрелая инженерная дисциплина постепенно вырабатывает собственный язык.

Это позволяет специалистам быстрее понимать друг друга и обсуждать сложные идеи без длинных объяснений.

DDD выполняет именно эту роль.

Он предлагает терминологию, которая позволяет архитекторам, аналитикам и разработчикам говорить об одних и тех же явлениях одинаковыми словами.

Когда архитектор произносит «предметная область», «единый язык» или «ограниченный контекст», он использует не модные термины, а профессиональные понятия, за которыми стоят конкретные инженерные идеи.

Таким образом, DDD не создает новое мышление.

Он помогает его точно выразить.

DDD помогает сохранить модель

Любая архитектура со временем развивается.

Меняются требования.

Появляются новые функции.

Команда разработки растет.

Приходят новые специалисты.

Именно в этот момент возникает главный риск.

Постепенно программная система начинает отходить от модели предметной области.

Названия становятся случайными.

Границы размываются.

Появляются технические сокращения, понятные только разработчикам.

Через несколько лет программа уже перестает отражать бизнес.

DDD предлагает инструменты, позволяющие замедлить этот процесс.

Он помогает сохранить связь между предметной областью и программной системой даже тогда, когда проект становится большим и сложным.

DDD не является обязательным рецептом

За годы существования вокруг DDD сформировалось множество рекомендаций, шаблонов и практик.

Некоторые из них прекрасно работают в крупных проектах.

Другие оказываются избыточными для небольших систем.

Поэтому архитектор не должен воспринимать DDD как обязательный набор правил.

Любая методология создается для того, чтобы помогать принимать решения, а не заменять профессиональное мышление.

Архитектор использует те инструменты, которые действительно решают задачи конкретного проекта.

Остальные могут так и остаться невостребованными.

Главное наследие DDD

Если попытаться сформулировать главную ценность предметно-ориентированного проектирования одной мыслью, то она будет звучать очень просто.

Программа должна отражать предметную область, а не скрывать ее.

Именно эта идея проходит через всю книгу.

Мы пришли к ней задолго до знакомства с терминологией DDD.

Поэтому теперь читатель может воспринимать DDD не как новую теорию, а как естественное продолжение уже сформировавшегося архитектурного мышления.

Вместо заключения

Предметно-ориентированное проектирование не делает человека архитектором.

Архитектором делает способность понимать бизнес, видеть существенные закономерности, строить модели и принимать инженерные решения.

DDD лишь помогает сохранить результаты этой работы в программной системе.

Поэтому ценность DDD заключается не в количестве терминов или шаблонов.

Его настоящая ценность — в том, что он позволяет программе на протяжении многих лет оставаться отражением предметной области, для которой она была создана.

Обычно DDD преподносят как центральную тему, которую читателю предстоит изучить. В нашей книге он появляется только после того, как читатель уже самостоятельно пришел к большинству ключевых идей. Благодаря этому возникает совсем другой эффект: DDD перестает восприниматься как сложная методология и становится профессиональным языком, который позволяет архитекторам точно описывать уже понятные им принципы. Этой мыслью я завершаю первую большую линию книги — от понимания бизнеса к программной модели — и одновременно подготавливаю читателя к следующей части, где речь пойдет уже не о мышлении, а о построении архитектуры программной системы

Часть V. Проектирование программной архитектуры

Глава 24. Как архитектор выбирает архитектурный стиль

Архитектурный стиль не бывает правильным или неправильным

К этому моменту архитектор уже понимает предметную область.

Построена программная модель.

Определены основные понятия.

Выделены границы модулей.

Теперь необходимо принять решение, которое во многом определит дальнейшее развитие всей системы.

Какой архитектурный стиль выбрать?

На первый взгляд может показаться, что существует архитектурный стиль, превосходящий все остальные.

На практике это не так.

Каждый стиль проектирования создавался для решения определенного круга задач.

Поэтому архитектор не ищет «лучшую» архитектуру.

Он ищет архитектуру, наиболее соответствующую требованиям конкретной системы.

Архитектура программы начинается с вопросов, а что поделаешь - снова они?

Начинающие специалисты нередко начинают обсуждение будущей системы с названий технологий.

«Будут ли использоваться микросервисы?»

«Нужна ли событийная архитектура?»

«Следует ли строить сервис-ориентированную систему?»

Для архитектора подобные вопросы возникают слишком рано.

Сначала он пытается понять характер будущей системы.

Насколько она сложна?

Как быстро будет изменяться?

Сколько команд будут одновременно работать над проектом?

Какие требования предъявляются к надежности?

Каковы ожидаемые объемы данных?

Какие части системы должны масштабироваться независимо?

Какие компоненты обязаны работать автономно?

Только после ответа на эти вопросы появляется возможность обсуждать архитектурный стиль.

Архитектурный стиль — это компромисс

Любое архитектурное решение обладает достоинствами и ограничениями.

Выбирая одно свойство системы, архитектор почти всегда отказывается от другого.

Простая архитектура обычно легче в сопровождении.

Распределенная архитектура обеспечивает большую гибкость, но требует значительно более сложного взаимодействия между компонентами.

Высокая независимость модулей повышает автономность команд разработки, однако усложняет обмен данными и управление согласованностью.

Поэтому выбор архитектурного стиля всегда представляет собой поиск разумного компромисса между противоречивыми требованиями.

Монолит — это не ошибка

В профессиональной среде нередко можно встретить мнение, что современная система обязательно должна строиться на основе микросервисной архитектуры.

Подобный подход является заблуждением.

Монолитная архитектура остается одним из наиболее эффективных решений для огромного количества проектов.

Монолит проще? — Нет! Монолит мы выбираем не потому, что он "простой", а потому что его внутренняя структура может быть более понятной и связной, чем у распределенной системы, на определенном этапе.

Проще в тестировании.

Проще в сопровождении.

Проще в развертывании.

Во многих случаях именно монолит позволяет быстрее получить устойчивую и качественную систему.

Архитектор отказывается от него только тогда, когда появляются объективные причины.

Не раньше.

Модульность важнее распределенности

По мере роста системы становится очевидно, что основная проблема заключается не в количестве серверов.

Главная проблема — сложность самой программы.

Поэтому следующим естественным шагом становится разделение системы на независимые программные модули.

Если границы модулей определены правильно, такая архитектура способна развиваться многие годы без перехода к распределенной системе.

Именно поэтому многие современные проекты сначала становятся модульными монолитами и лишь затем, при необходимости, эволюционируют в сторону микросервисов.

Микросервисы появляются не первыми

Если система продолжает расти, увеличивается количество команд, появляются независимые циклы развития различных подсистем, архитектор начинает рассматривать возможность их физического разделения.

Именно в этот момент может возникнуть микросервисная архитектура.

Важно понимать, что микросервисы не являются целью проектирования.

Они становятся следствием требований к независимому развитию отдельных частей системы.

Если подобных требований нет, искусственное разделение программы лишь увеличивает ее сложность.

События и сервисы — это способы взаимодействия

После выбора общей архитектуры возникает следующий вопрос.

Как отдельные части системы будут обмениваться информацией?

В одних случаях достаточно непосредственного вызова программного интерфейса.

В других оказывается предпочтительным обмен событиями.

Иногда необходима сервис-ориентированная архитектура.

Иногда — сочетание нескольких подходов.

И здесь архитектор снова принимает решение, исходя не из популярности технологий, а из характера взаимодействия между подсистемами.

Архитектурный стиль определяет не используемые инструменты, а способ организации сотрудничества между частями системы.

Архитектор выбирает не технологии

Со стороны может показаться, что архитектор принимает решение между монолитом, микросервисами или событийной архитектурой.

На самом деле он отвечает совсем на другой вопрос.

Какой должна быть организация системы, чтобы она соответствовала требованиям бизнеса сегодня и могла изменяться вместе с ним завтра?

Ответ на этот вопрос и определяет архитектурный стиль.

Все остальные решения появляются значительно позже.

Вместо заключения

Архитектурный стиль нельзя выбрать по моде, рекомендациям конференций или популярности технологий.

Он рождается как следствие требований к системе.

Чем глубже архитектор понимает предметную область, характер будущей эксплуатации и ожидаемое развитие проекта, тем осознаннее становится его выбор.

Поэтому опытный архитектор начинает не с вопроса «Какая архитектура лучше?», а с вопроса «Какая архитектура лучше всего соответствует этой системе?»

Мне кажется, именно здесь стоит оставить читателя с одной мыслью

Архитектурный стиль — это не технология и не цель проектирования. Это инженерный ответ на требования конкретной системы.

Эта глава важна тем, что меняет угол зрения. После первых четырех частей читатель уже знает, как понимать бизнес и строить программную модель. Теперь он впервые сталкивается с необходимостью выбирать между различными архитектурными подходами. И ключевая идея главы заключается не в сравнении монолита и микросервисов, а в понимании самого процесса выбора: архитектор никогда не начинает с ответа. Он начинает с вопросов, а архитектурный стиль становится их логическим следствием. Именно

этот способ мышления отличает архитектора от разработчика, выбирающего технологию по привычке или популярности.

Глава 25. Пример выбора архитектуры приложения

1. Монолит

Самый простой вариант.

Вся система представляет собой одно приложение.

Например:

Интернет-магазин

Одно приложение

```
|------------------------|
| Каталог                |
| Корзина                |
| Заказы                 |
| Оплата                 |
| Пользователи           |
-------------------------
PostgreSQL
```

Запускается один процесс.

Один Docker-контейнер.

Одна база данных (не обязательно, но чаще всего).

Один деплой.

Все находится внутри одного приложения.

Преимущества

- очень просто разрабатывать;
- просто тестировать;
- просто сопровождать;
- минимальные накладные расходы.

Недостаток

По мере роста системы код начинает сильно переплетаться.

2. Модульный монолит

Вот здесь начинается настоящая архитектура.

Физически приложение по-прежнему одно.

Но внутри него существуют строгие модули.

Например

Приложение

Каталог

Заказы

Платежи

CRM

Склад

Уведомления

Каждый модуль:

- имеет собственную ответственность;
- имеет собственную модель;
- имеет собственные интерфейсы;
- не знает внутреннего устройства соседнего модуля.

То есть внутри программы появляются почти такие же границы, как между будущими микросервисами.

Но приложение остается единым.

Представим структуру проекта.

src/

Catalog/

Orders/

Warehouse/

Payments/

CRM/

Notifications/

Каждый каталог — практически отдельная система.

Но запускаются они одним процессом.

Именно поэтому модульный монолит считается очень сильной архитектурой.

Он позволяет получить почти все преимущества микросервисов.

Но без их сложности.

3. Микросервисы

Следующий этап.

Теперь каждый модуль становится самостоятельным приложением.

Например

Catalog Service

Orders Service

Warehouse Service

Payments Service

CRM Service

Notification Service

Каждый сервис:

- имеет собственный процесс;
- может иметь собственную базу данных;
- разворачивается независимо;
- обновляется независимо;

● масштабируется независимо.

Между ними уже нет вызовов методов.

Есть:

REST

gRPC

Kafka

RabbitMQ

HTTP

События

Схематично

API

|

Catalog

Orders

Warehouse

Payments

CRM

Notification

Почему многие начинают именно с модульного монолита

Потому что он позволяет проверить архитектуру.

Если внутри одного приложения невозможно правильно разделить систему на модули,

то после разделения на микросервисы проблемы станут только хуже.

Поэтому опытные архитекторы сначала строят хорошие внутренние границы.

И только потом делают физическое разделение.

Очень хорошая аналогия

Представь большой офис.

Монолит

Все сотрудники работают в одном огромном помещении.

_____

Все сидят вместе

_____

Модульный монолит

Теперь появились отделы.

_____

Продажи

|

Логистика

|

Финансы

|

Поддержка

_____

Все находятся в одном здании.

Но у каждого отдела свои задачи.

Микросервисы

Теперь каждый отдел переехал в собственное здание.

Продажи

|

Логистика

|

Финансы

|

Поддержка

Появляются:

телефон,почта,

курьеры,

регламенты,

совещания.

То есть взаимодействие становится намного сложнее.

И вот здесь рождается главный вывод

Мне кажется, именно его стоит здесь написать..

Микро сервисы не делают архитектуру лучше. Они лишь меняют способ взаимодействия между уже существующими частями системы. Если внутри монолита отсутствуют четкие границы между модулями, переход к микро сервисам не решит эту проблему. Напротив, он сделает ее значительно дороже и сложнее в сопровождении.

Именно поэтому многие опытные архитекторы сегодня рекомендуют начинать с модульного монолита. Сначала нужно научиться проектировать правильные границы, зависимости и ответственность внутри одного приложения. И лишь когда эти границы становятся устойчивыми, а требования бизнеса действительно требуют независимого развертывания, масштабирования или развития отдельных частей системы, возникает естественный переход к микро сервисной архитектуре. Это не шаг «вперед» по шкале технологий, а следующий этап эволюции системы, обусловленный ее реальными потребностями.

Глава 26. Как проектируются программные модули

Архитектура состоит не из классов

К этому моменту архитектор уже знает, какой должна быть система.

Определен архитектурный стиль.

Понятна предметная область.

Выделены основные части бизнеса.

Теперь возникает новая задача.

Необходимо определить внутреннее устройство программы.

На этом этапе многие разработчики начинают проектировать классы, интерфейсы, базы данных и каталоги проекта.

Архитектор поступает иначе.

Сначала он определяет модули.

И лишь затем решает, какие классы будут находиться внутри каждого из них.

Программный модуль становится основной единицей проектирования.

Не файл.

Не класс.

Не пакет.

Именно модуль.

Что такое программный модуль

Под программным модулем понимается самостоятельная часть системы, обладающая собственной ответственностью, внутренней логикой и четко определенными границами.

Важно понимать, что модуль — это не каталог проекта и не пространство имен.

Это логическая часть системы.

Он объединяет все, что необходимо для решения одной группы задач, и скрывает внутреннее устройство от остальных частей программы.

Размер модуля не имеет принципиального значения.

Он может состоять из нескольких классов.

Может включать сотни файлов.

Главное — чтобы модуль представлял собой единую законченную область ответственности.

Ответственность определяет границы

Главный вопрос, который задает архитектор при проектировании модуля, звучит неожиданно просто:

За что отвечает этот модуль?

Не:

Какие классы в него поместить?

Не:

Какие таблицы базы данных он использует?

А именно:

Какую задачу бизнеса он решает?

Если ответить на этот вопрос невозможно, значит границы модуля определены неверно.

Хороший модуль всегда можно описать одним предложением.

Например:

«Этот модуль отвечает за регистрацию пользователей.»

Или:

«Этот модуль отвечает за обработку заказов.»

Если описание превращается в длинный перечень несвязанных обязанностей, значит модуль объединяет несколько различных областей ответственности и требует дальнейшей декомпозиции.

Хороший модуль знает о себе все

Архитектор стремится к тому, чтобы вся логика, относящаяся к одной области ответственности, находилась внутри одного модуля.

Если правила регистрации пользователей разбросаны по десяти различным каталогам проекта, то архитектура постепенно начинает разрушаться.

Поэтому модуль стремится стать «хозяином» своей области.

Именно внутри него сосредоточены:

- модели;
- бизнес-правила;
- прикладные сценарии;
- проверки;
- преобразования данных;
- внутренние сервисы.

Все, что относится к его ответственности.

Публичный интерфейс важнее внутреннего устройства

Ни один модуль не существует изолированно.

Остальная система должна иметь возможность взаимодействовать с ним.

Но взаимодействовать следует не с внутренними деталями реализации, а только через специально определенный интерфейс.

Архитектор проектирует модуль таким образом, чтобы внешнему миру были доступны только необходимые точки взаимодействия.

Все остальные детали остаются скрытыми.

Такой подход позволяет изменять внутреннее устройство модуля, не затрагивая остальные части системы.

Чем меньше окружающий код знает о внутреннем устройстве модуля, тем проще становится дальнейшее развитие системы.

Скрытие реализации — основа устойчивой архитектуры

Одной из главных причин высокой стоимости сопровождения становится чрезмерная открытость модулей.

Если любой разработчик может обращаться к внутренним классам, изменять состояние объектов или использовать вспомогательные методы напрямую, архитектурные границы постепенно исчезают.

Через некоторое время система превращается в сеть случайных зависимостей.

Поэтому архитектор сознательно ограничивает доступ к внутреннему устройству каждого модуля.

Изменения должны происходить внутри модуля, не оказывая влияния на остальные части программы.

Зависимости должны быть осознанными

После появления нескольких модулей возникает следующий вопрос.

Как они будут взаимодействовать друг с другом?

Каждая новая зависимость увеличивает сложность системы.

Поэтому архитектор рассматривает зависимость не как удобство программирования, а как инженерное решение.

Каждая зависимость должна иметь понятную причину существования.

Если модуль начинает использовать большое количество других модулей, это почти всегда является признаком неудачной декомпозиции.

Чем проще граф зависимостей, тем легче сопровождать систему.

Высокая связность и слабое зацепление

В инженерной литературе существуют два принципа, которые на протяжении многих десятилетий считаются основой качественного проектирования.

Высокая связность означает, что элементы внутри одного модуля тесно связаны между собой и работают над решением одной задачи.

Все, что находится внутри модуля, существует ради его основной ответственности.

Слабое зацепление означает, что различные модули знают друг о друге как можно меньше.

Они взаимодействуют только через заранее определенные интерфейсы и не зависят от внутреннего устройства друг друга.

Именно сочетание этих двух свойств делает архитектуру устойчивой к изменениям.

Изменение одного модуля редко приводит к необходимости изменять остальные части системы.

Архитектор проектирует не файлы, а границы

Многие начинающие разработчики воспринимают архитектуру как способ красивого размещения файлов по каталогам.

На практике архитектура начинается значительно раньше.

Архитектор определяет:

- где заканчивается ответственность одного модуля;
- где начинается ответственность другого;
- каким образом они взаимодействуют;
- какие знания могут покидать пределы модуля, а какие должны оставаться внутри.

Лишь после этого появляются каталоги, пространства имен, классы и исходный код.

Вместо заключения

Программная архитектура строится не из классов и не из технологий.

Она строится из правильно выделенных модулей.

Если границы модулей определены удачно, система способна развиваться долгие годы, независимо от используемых языков программирования, фреймворков или способов развертывания.

Именно поэтому опытный архитектор начинает проектирование не с написания кода, а с поиска естественных границ будущей программы.

Я думаю,, в этой главе появилась одна мысль, которая хорошо связывает всю книгу: в предыдущих частях мы искали границы в бизнесе, а теперь ищем границы в программе. Это не две разные задачи, а две стороны одной инженерной работы. Если архитектор правильно увидел границы предметной области, то именно они становятся основой для проектирования программных модулей. В этом смысле декомпозиция программы не начинается заново — она является продолжением той работы, которая была выполнена при изучении бизнеса и построении программной модели.

Глава 27. Как проектируются взаимодействия между модулями

Хорошие модули должны уметь сотрудничать

После того как архитектор определил границы программных модулей, работа не заканчивается.

Наоборот, появляется новая задача.

Отдельные части системы должны работать совместно.

Ни один модуль не существует сам по себе.

Модуль регистрации пользователей должен передавать информацию модулю безопасности.

Модуль заказов взаимодействует со складом.

Платежная система сообщает о проведенной оплате.

Служба уведомлений отправляет сообщения пользователям.

Архитектор начинает проектировать уже не сами модули, а способы их взаимодействия.

Именно здесь рождается архитектура системы как единого целого.

Взаимодействие — это договор

Очень часто взаимодействие между модулями воспринимается как обычный вызов метода или функции.

Для архитектора это гораздо больше.

Каждое взаимодействие представляет собой договор между двумя частями системы.

Один модуль принимает на себя определенные обязательства.

Другой модуль рассчитывает, что эти обязательства будут выполнены.

Поэтому архитектор проектирует не отдельные вызовы, а правила сотрудничества между независимыми частями программы.

Чем проще и понятнее эти правила, тем устойчивее становится вся система.

Не каждый обмен информацией одинаков

В процессе проектирования становится очевидно, что разные задачи требуют разных способов взаимодействия.

Иногда системе необходимо выполнить действие немедленно.

Иногда достаточно лишь сообщить, что некоторое событие произошло.

Иногда требуется получить данные.

Иногда — инициировать выполнение операции.

Хотя внешне все это выглядит как обмен информацией, с инженерной точки зрения это разные виды взаимодействий.

Понимание этой разницы позволяет избежать множества архитектурных ошибок.

Команда — просьба выполнить действие

Команда появляется тогда, когда один модуль просит другой выполнить определенную работу.

Например:

- зарегистрировать пользователя;
- создать заказ;
- сформировать счет;
- отправить письмо.

Команда предполагает ответственность за выполнение действия.

Получатель команды принимает решение, каким образом выполнить поставленную задачу.

Запрос — получение информации

Иногда модулю не требуется инициировать действие.

Ему необходимо лишь получить сведения.

Например:

- найти пользователя;

- определить статус заказа;
- получить список товаров;
- узнать остаток на складе.

Такие взаимодействия называются запросами.

Запрос не изменяет состояние системы.

Он лишь предоставляет информацию, необходимую для дальнейшей работы.

Событие — сообщение о произошедшем

Существует и другой способ взаимодействия.

Модуль ничего не просит выполнить.

Он просто сообщает окружающим, что некоторое событие уже произошло.

Например:

- пользователь зарегистрирован;
- заказ оформлен;
- платеж выполнен;
- товар поступил на склад.

Важно понимать принципиальную разницу.

Команда направлена в будущее.

Событие описывает уже свершившийся факт.

Это различие оказывает огромное влияние на архитектуру всей системы.

Синхронное и асинхронное взаимодействие

После выбора типа взаимодействия архитектор принимает еще одно решение.

Должен ли отправитель ждать результата?

Если ответ требуется немедленно, используется синхронное взаимодействие.

Один модуль обращается к другому и продолжает работу только после получения ответа.

Такой подход прост и понятен.

Однако он делает систему более зависимой от доступности всех участников взаимодействия.

Если же ожидание результата не требуется, архитектор может выбрать асинхронный способ.

В этом случае модуль передает сообщение и продолжает собственную работу.

Получатель обработает его тогда, когда сможет.

Асинхронное взаимодействие позволяет строить более устойчивые и масштабируемые системы.

Но одновременно усложняет проектирование и сопровождение.

Контракт важнее реализации

Модули должны знать друг о друге как можно меньше.

Поэтому архитектор стремится к тому, чтобы взаимодействие происходило через заранее определенные контракты.

Контракт определяет:

- какие сообщения могут быть переданы;
- какие данные содержит каждое сообщение;
- какие гарантии предоставляет принимающая сторона;
- какие ошибки могут возникнуть;
- какие результаты ожидаются.

При этом внутреннее устройство модуля остается неизвестным окружающим.

Такое разделение позволяет изменять реализацию, не нарушая работу всей системы.

Простота взаимодействий — признак зрелой архитектуры

Сложность системы определяется не количеством модулей.

Она определяется количеством связей между ними.

Чем больше прямых зависимостей возникает, тем труднее становится развитие программы.

Опытный архитектор стремится проектировать взаимодействия таким образом, чтобы каждый модуль сотрудничал только с теми частями системы, без которых действительно невозможно выполнить его задачу.

Избыточные связи делают архитектуру хрупкой.

Минимально необходимые — позволяют ей развиваться.

Архитектор проектирует сотрудничество

Многие считают, что архитектура заканчивается после определения структуры программы.

На практике именно в этот момент начинается наиболее сложная работа.

Недостаточно разделить систему на качественные модули.

Необходимо сделать так, чтобы они могли эффективно сотрудничать.

Каждое взаимодействие должно быть понятным.

Каждый контракт — устойчивым.

Каждая зависимость — осознанной.

Именно из множества таких решений постепенно рождается архитектура, способная выдерживать развитие системы на протяжении многих лет.

Вместо заключения

Программные модули редко работают изолированно.

Они постоянно обмениваются данными, инициируют действия и реагируют на происходящие события.

От того, насколько грамотно спроектированы эти взаимодействия, зависит не только производительность системы, но и ее понятность, сопровождаемость и способность к развитию.

Архитектор проектирует не отдельные вызовы функций и не сетевые запросы.

Он проектирует сотрудничество между независимыми частями системы.

Именно это сотрудничество превращает набор модулей в единую работающую архитектуру.

В главе 26 архитектор отвечает на вопрос «как разделить программу?», а здесь — «как сделать так, чтобы разделенные части снова стали единым целым?». Это естественный переход от декомпозиции к интеграции. И именно на этом этапе читатель начинает понимать, что архитектура — это не только умение разделять систему, но и умение правильно организовать взаимодействие между ее частями. Именно здесь инженерное мышление архитектора проявляется в полной мере.

Глава 28. Как архитектор выбирает технологии

Технологии не являются архитектурой

К этому моменту система уже существует.

Еще нет ни одной строки программного кода.

Не выбран язык программирования.

Не выбрана база данных.

Не определен брокер сообщений.

Но архитектура уже появилась.

Известна предметная область.

Построена программная модель.

Определены программные модули.

Спроектированы взаимодействия между ними.

Выбран архитектурный стиль.

Другими словами, архитектор уже понимает, что именно должна делать система.

Теперь остается ответить на следующий вопрос.

Какими средствами это лучше реализовать?

Именно здесь впервые появляются технологии.

Но не как отправная точка проектирования.

А как инструмент реализации уже принятых архитектурных решений.

Архитектор выбирает не технологии

Со стороны может показаться, что архитектор выбирает между PostgreSQL и MongoDB, между Kafka и RabbitMQ или между Redis и Memcached.

На практике происходит совершенно другой процесс.

Архитектор никогда не начинает с названия продукта.

Он начинает с инженерной задачи.

Необходимо надежно хранить данные.

Необходимо обеспечить быстрый доступ к часто используемой информации.

Необходимо организовать обмен сообщениями между независимыми частями системы.

Необходимо выполнять длительные операции в фоновом режиме.

Необходимо предоставить внешним системам программный интерфейс.

Каждая такая задача требует определенного архитектурного решения.

И только после этого начинается выбор технологии, способной реализовать это решение.

Каждая технология отвечает на свой вопрос

За годы развития программной инженерии появились тысячи инструментов.

Но каждый из них создавался для решения вполне определенного класса задач.

Реляционные базы данных предназначены для надежного хранения взаимосвязанных данных.

Системы кэширования позволяют уменьшить время доступа к информации.

Брокеры сообщений обеспечивают обмен сообщениями между независимыми компонентами.

Поисковые движки ускоряют поиск по большим объемам информации.

Объектные хранилища предназначены для работы с файлами.

Балансировщики нагрузки распределяют запросы между экземплярами приложения.

Технологии отличаются друг от друга не потому, что одни лучше других.

Они отличаются тем, какие задачи помогают решать.

Сначала решение — потом инструмент

Опытный архитектор редко задает вопрос:

«Какую базу данных выбрать?»

Гораздо чаще он спрашивает:

«Как должны храниться данные этой системы?»

После этого становится понятно, нужна ли реляционная модель данных, документоориентированное хранилище, графовая база данных или комбинация нескольких решений.

Точно так же архитектор не спрашивает:

«Использовать Kafka или RabbitMQ?»

Сначала он определяет характер взаимодействия между модулями.

Нужна ли доставка каждого сообщения?

Необходимо ли хранить историю событий?

Требуется ли обработка потоков данных в реальном времени?

Только после ответа на эти вопросы становится возможным осознанный выбор конкретной технологии.

Не существует универсальных технологий

Начинающие разработчики нередко ищут «лучшую» базу данных, «лучший» брокер сообщений или «лучший» язык программирования.

Архитектор понимает, что подобных решений не существует.

Каждая технология обладает сильными и слабыми сторонами.

Одни лучше работают с транзакциями.

Другие — с большими объемами данных.

Одни обеспечивают минимальную задержку.

Другие — высокую надежность.

Одни упрощают сопровождение.

Другие позволяют эффективнее масштабировать систему.

Поэтому выбор технологии всегда представляет собой компромисс между требованиями проекта.

Популярность не является аргументом

Индустрия программного обеспечения постоянно меняется.

Каждый год появляются новые платформы, базы данных, брокеры сообщений и облачные сервисы.

Некоторые из них становятся чрезвычайно популярными.

Однако популярность сама по себе ничего не говорит об их пригодности для конкретной системы.

Архитектор не выбирает технологии потому, что они находятся на пике интереса сообщества.

Он выбирает их потому, что они помогают решить конкретную инженерную задачу.

Если задача изменилась, может измениться и выбор технологии.

Хорошая архитектура переживает смену технологий

За время жизни информационной системы технологии неизбежно меняются.

Появляются новые версии языков программирования.

Устаревают библиотеки.

Меняются облачные платформы.

Появляются новые способы хранения данных и обмена сообщениями.

Если архитектура слишком тесно связана с конкретной технологией, подобные изменения становятся чрезвычайно дорогими.

Поэтому опытный архитектор стремится строить систему таким образом, чтобы замена технологии не приводила к пересмотру архитектурных решений.

Технологии должны быть сменяемыми.

Архитектура — устойчивой.

Архитектор принимает инженерные решения

Можно сказать, что архитектор выбирает технологии.

Но это будет не совсем точно.

На самом деле он выбирает способы решения инженерных задач.

- Как хранить данные?
- Как организовать взаимодействие между модулями?
- Как обеспечить надежность?
- Как масштабировать систему?
- Как выполнять длительные операции?
- Как обеспечить безопасность?

Когда ответы на эти вопросы найдены, выбор конкретных технологий становится значительно проще.

Во многих случаях оказывается, что несколько различных инструментов способны одинаково успешно реализовать одно и то же архитектурное решение.

Вместо заключения

Технологии постоянно появляются и исчезают.

Одни становятся популярными, другие уходят в историю.

Архитектурные принципы меняются значительно медленнее.

Именно поэтому архитектор начинает не с выбора инструментов, а с понимания задач, которые предстоит решить.

Когда архитектурное решение уже принято, технологии перестают быть предметом спора.

Они становятся средствами достижения поставленной цели.

Именно в этом заключается одно из главных отличий архитектора от разработчика.

Разработчик чаще задает вопрос:

«Какую технологию использовать?»

Архитектор сначала спрашивает:

«Какую инженерную задачу необходимо решить?»

И лишь после этого выбирает инструмент, который позволит сделать это наиболее эффективно.

Я последовательно показывал читателю, что архитектор начинает с бизнеса, затем строит модель предметной области, проектирует программную модель, определяет архитектурный стиль, декомпозирует систему на модули и проектирует их взаимодействие. Только после этого возникает вопрос о выборе технологий. Таким образом, технология перестает быть отправной точкой проектирования и занимает свое естественное место — она становится способом реализации уже принятых инженерных решений. Именно эта последовательность, на мой взгляд, отличает архитектурное мышление от простого выбора популярных инструментов.

Глава 29. Как проектируется инфраструктура системы

Программа не существует сама по себе

К этому моменту работа над программной архитектурой практически завершена.

Определены модули.

Спроектированы взаимодействия.

Выбраны технологии.

Может показаться, что систему уже можно считать законченной.

Но это лишь начало.

Любая программа существует не в исходных текстах.

Она существует в работающей среде.

Именно там ее используют пользователи.

Именно там происходят сбои.

Именно там возникают реальные нагрузки.

Поэтому следующая задача архитектора состоит в проектировании не самой программы, а среды, в которой ей предстоит работать.

Инфраструктура — часть архитектуры

Долгое время инфраструктуру воспринимали как отдельную область, не связанную с архитектурой программного обеспечения.

Считалось, что сначала разработчики создают программу, а затем системные администраторы обеспечивают ее работу.

Современные информационные системы давно вышли за рамки такого подхода.

Сегодня архитектурные решения невозможно отделить от инфраструктуры.

Архитектор должен понимать, каким образом будет развертываться система, как она будет масштабироваться, каким образом будут

храниться настройки, где будут находиться секреты, как обнаруживать ошибки и как обеспечивать отказоустойчивость.

Инфраструктура становится естественным продолжением архитектуры программы.

Среда должна быть предсказуемой

Одной из первых задач становится создание одинаковых условий работы системы.

Разработка.

Тестирование.

Предварительная эксплуатация.

Промышленная среда.

Если приложение работает по-разному в каждом окружении, сопровождение быстро превращается в постоянный поиск случайных причин ошибок.

Поэтому архитектор стремится сделать среду максимально предсказуемой.

Именно эту задачу сегодня решает контейнеризация.

Контейнер позволяет переносить не только программу, но и всю среду ее выполнения.

Это снижает количество различий между окружениями и делает развертывание значительно более надежным.

Контейнеризация не является целью.

Она лишь помогает обеспечить воспроизводимость среды.

Архитектура должна учитывать отказ оборудования

Любая инфраструктура рано или поздно сталкивается с отказами.

Выходит из строя сервер.

Прерывается сетевое соединение.

Заканчивается место на диске.

Перезапускается виртуальная машина.

Архитектор исходит из того, что подобные события неизбежны.

Поэтому система должна продолжать работать даже в условиях частичных отказов.

Именно отсюда появляются решения, связанные с резервированием, балансировкой нагрузки, автоматическим восстановлением сервисов и распределением компонентов между несколькими узлами.

Отказоустойчивость никогда не появляется случайно.

Она проектируется заранее.

Масштабирование — следствие роста системы

Если система развивается успешно, со временем увеличивается количество пользователей, объем данных и нагрузка на отдельные компоненты.

Архитектор должен заранее понимать, каким образом система будет расти.

Иногда достаточно увеличить вычислительные ресурсы одного сервера.

Иногда возникает необходимость распределить нагрузку между несколькими экземплярами приложения.

Иногда разные части системы начинают масштабироваться независимо друг от друга.

Важно отметить, что масштабирование не является самоцелью.

Большинство проектов никогда не сталкивается с экстремальными нагрузками.

Поэтому архитектор проектирует масштабирование как ответ на реальные требования бизнеса, а не как демонстрацию технических возможностей.

Настройки не должны находиться в программе

Любая система зависит от множества параметров.

Адресов сервисов.

Строк подключения к базам данных.

Параметров безопасности.

Ограничений нагрузки.

Если подобные значения оказываются встроенными в исходный код, каждое изменение превращается в изменение программы.

Поэтому современная архитектура предполагает отделение конфигурации от реализации.

Настройки становятся частью среды выполнения.

Это позволяет изменять окружение без изменения самой программы.

Секреты требуют особого отношения

Пароли.

Ключи шифрования.

Токены доступа.

Сертификаты.

Все эти данные отличаются от обычной конфигурации.

Их утечка может привести к компрометации всей системы.

Поэтому архитектор рассматривает секреты как отдельный объект проектирования.

Они должны храниться, передаваться и обновляться по правилам, отличным от остальных параметров системы.

Безопасность начинается не с криптографических алгоритмов.

Она начинается с правильной организации работы с секретами.

Работающая система должна быть наблюдаемой

Если система перестает отвечать на запросы, пользователи замечают проблему мгновенно.

Архитектору же необходимо понять значительно больше.

Что произошло?

Когда это произошло?

Почему это произошло?

Какая часть системы стала причиной сбоя?

Ответить на эти вопросы невозможно без информации о работе самой системы.

Именно поэтому современные архитектуры предусматривают механизмы наблюдаемости.

Логирование позволяет восстановить последовательность событий.

Метрики показывают состояние системы во времени.

Трассировка помогает проследить путь выполнения одного запроса через несколько компонентов.

Наблюдаемость превращает сопровождение из поиска догадок в инженерный анализ.

Облака и платформы — это средство, а не цель

Сегодня большинство новых систем создается с использованием облачных платформ.

Появились инструменты автоматического развертывания, управления контейнерами и масштабирования приложений.

Одним из наиболее известных решений является Kubernetes.

Однако архитектор не начинает проектирование с вопроса:

«Будем ли мы использовать Kubernetes?»

Он сначала отвечает на другой вопрос:

«Какие задачи должна решать инфраструктура нашей системы?»

Лишь после этого становится понятно, требуется ли сложная платформа оркестрации, достаточно ли виртуальных машин или подойдет более простая схема развертывания.

Как и в предыдущих главах, технология становится следствием архитектурного решения, а не его причиной.

Вместо заключения

Программная архитектура не заканчивается написанием кода.

Она продолжается в той среде, где система будет работать многие годы.

Инфраструктура определяет, насколько надежной окажется система, как она будет развиваться, насколько быстро удастся обнаруживать проблемы и насколько уверенно она переживет рост нагрузки.

Именно поэтому архитектор проектирует не только программу.

Он проектирует условия ее существования.

И чем раньше инфраструктура становится частью архитектурного мышления, тем устойчивее оказывается вся система.

Мы начали с вопроса «что строить?», затем перешли к «как устроить программу?», а теперь отвечаем на вопрос «где и как эта программа будет жить?». Это естественное завершение проектирования: архитектура существует не на диаграммах и не в репозитории исходного кода. Ее настоящая проверка начинается в эксплуатации. Именно там становятся видны качество принятых решений, способность системы развиваться и готовность выдерживать реальные условия работы. Поэтому инфраструктура — это не приложение к архитектуре, а ее неотъемлемая часть.

Глава 30. Как архитектор проектирует развитие системы

Архитектура не заканчивается релизом

Многие представляют архитектуру как этап проекта.

Появляется идея.

Пишутся требования.

Проектируется система.

Разрабатывается программа.

Происходит внедрение.

После этого архитектура считается завершенной.

На практике все происходит иначе.

Именно после первого релиза архитектура начинает жить собственной жизнью.

Система начинает использоваться.

Появляются реальные пользователи.

Возникают новые требования.

Изменяются бизнес-процессы.

Меняется законодательство.

Появляются новые интеграции.

То, что вчера казалось удачным решением, через несколько лет может стать ограничением для дальнейшего развития.

Архитектор проектирует не только текущую систему.

Он проектирует ее способность изменяться.

Изменения неизбежны

Ни одна информационная система не остается неизменной.

Компания выходит на новые рынки.

Появляются новые продукты.

Меняются способы работы с клиентами.

Изменяются требования безопасности.

Обновляются нормативные документы.

Развитие бизнеса неизбежно приводит к развитию программной системы.

Поэтому хороший архитектор никогда не рассчитывает, что требования однажды перестанут изменяться.

Наоборот.

Он исходит из того, что изменения являются естественным состоянием любой сложной системы.

Архитектура должна облегчать изменения

Если каждое новое требование требует переработки половины программы, проблема заключается не в требованиях.

Проблема заключается в архитектуре.

Одной из главных целей архитектурных решений является локализация изменений.

Изменение одной бизнес-функции не должно приводить к каскадному изменению всей системы.

Именно поэтому ранее мы так много внимания уделяли границам предметной области, программным модулям и взаимодействию между ними.

Все эти решения принимаются не ради красоты архитектуры.

Они принимаются ради будущих изменений.

Технический долг возникает не только из-за плохого кода

Технический долг часто связывают с нарушением правил программирования.

Однако архитектурный долг может оказаться значительно опаснее.

Он возникает тогда, когда первоначальные архитектурные решения перестают соответствовать новым условиям, но продолжают использоваться.

Иногда это происходит осознанно.

Необходимо быстрее вывести продукт на рынок.

Не хватает времени.

Ограничен бюджет.

Иногда причины связаны с изменением самого бизнеса.

В любом случае архитектор должен понимать, какие решения были временным компромиссом, а какие должны оставаться основой системы.

Технический долг становится опасным не тогда, когда он появляется.

Он становится опасным тогда, когда о нем перестают помнить.

Развитие требует бережного отношения к существующей системе

Со временем любая система начинает взаимодействовать с другими системами.

Появляются внешние пользователи.

Интеграции.

Мобильные приложения.

Партнерские сервисы.

Любое изменение может повлиять на десятки зависимых компонентов.

Поэтому архитектор должен учитывать обратную совместимость.

Новые возможности не должны без необходимости нарушать работу уже существующих клиентов системы.

Иногда сохранить совместимость невозможно.

Но в таких случаях изменения должны быть заранее спланированы, а переход — понятен и управляем.

Некоторые изменения требуют миграции

Иногда развитие системы невозможно без изменения уже существующих данных или способов их хранения.

Появляются новые структуры данных.

Изменяются форматы сообщений.

Разделяются программные модули.

Обновляются интерфейсы взаимодействия.

Подобные изменения редко происходят мгновенно.

Архитектор должен предусмотреть период, в течение которого старая и новая модели будут существовать одновременно.

Именно поэтому миграция представляет собой не разовую операцию, а отдельный архитектурный процесс.

Эволюция лучше революции

В инженерной практике нередко возникает соблазн полностью переписать существующую систему.

Подобные решения кажутся привлекательными.

Можно избавиться от накопленных ограничений.

Использовать современные технологии.

Построить архитектуру «с чистого листа».

На практике подобные проекты связаны с высоким риском.

Пока новая система создается, существующая продолжает работать, развиваться и изменяться.

В результате к моменту завершения переписывания новая версия нередко уже начинает отставать от реального бизнеса.

Поэтому опытные архитекторы предпочитают эволюционное развитие.

Система изменяется постепенно.

Небольшими, управляемыми шагами.

Каждый следующий этап опирается на предыдущий.

Такой подход позволяет непрерывно развивать систему, не останавливая ее работу.

Архитектор проектирует будущее

Хорошая архитектура отвечает не только на вопросы настоящего.

Она оставляет место для будущих решений.

Архитектор не способен предсказать все изменения, которые произойдут через пять или десять лет.

Но он способен создать систему, которая сможет эти изменения принять.

Именно поэтому зрелая архитектура ценится не за количество используемых технологий.

Она ценится за способность развиваться вместе с бизнесом.

Вместо заключения

На протяжении всей книги мы постепенно проходили путь архитектора.

Мы начали с понимания бизнеса.

Научились анализировать требования.

Определили предметную область.

Построили программную модель.

Выбрали архитектурный стиль.

Разделили систему на модули.

Спроектировали их взаимодействие.

Подобрали технологии.

Продумали инфраструктуру.

Теперь становится очевидно, что ни одно из этих решений не является окончательным.

Архитектура не существует в моменте.

Она существует во времени.

Ее качество определяется не тем, насколько удачно она выглядит в день первого релиза, а тем, насколько уверенно она позволяет системе развиваться спустя годы.

Именно поэтому архитектор проектирует не только программу.

Он проектирует ее будущее.

Мне кажется, эта глава удачно завершает пятую часть и подводит итог всему пути, который прошел читатель. Но я бы предложил одну мысль, которая может стать своеобразным мостом ко второй книге.

До этого момента мы отвечали на вопрос: «Как построить хорошую архитектуру?»

Во второй книге логично перейти к другому вопросу:

«Как сделать так, чтобы эта архитектура продолжала оставаться хорошей?»

Но это уже другая история.